



UNIVERSITY OF
PLYMOUTH

PUSL3190 Computing Project

Final Report

Personalized Sleep Disorder Prediction Using Advanced Machine Learning Techniques

Supervisor: Ms. Thilini Bakmeedeniya

Name: Warnakulasuriya Kurera

Plymouth Index Number: 10953721

Degree Program: BSc (Hons) Data Science

Acknowledgements

The successful completion of this final year computing project on *Sleep Disorder Prediction using Machine Learning and a Web-Based System* would not have been possible without the guidance, support, and encouragement of numerous individuals, to whom I wish to express my sincere gratitude.

I extend my heartfelt appreciation to the Dean of the Faculty of Computing, Dr. Rasika Ranaweera, and the Head of Module, Mr. Diluka Wijesinghe, along with all academic staff of NSBM Green University, for their dedicated coordination, knowledge sharing, and continuous support throughout the module and my broader academic journey. Their collective mentorship and unwavering commitment to academic excellence provided a strong foundation for the successful execution of this project.

My deepest gratitude is extended to my Supervisor, Ms. Thilini Bakmeedeniya, whose expert guidance, constructive feedback, and steadfast encouragement proved invaluable at every stage of this research. Her mentorship was instrumental in shaping both the technical depth and academic integrity of this work.

I also wish to acknowledge my fellow batchmates and peers for their collaborative spirit, intellectual camaraderie, and motivational support, which greatly contributed to sustaining my progress throughout this project.

Finally, I am profoundly grateful to my parents for their unconditional love, unwavering encouragement, and immeasurable sacrifices, which have served as my greatest source of strength and inspiration throughout this academic endeavor.

Abstract

Sleep disorders such as insomnia and sleep apnea are increasingly recognized as major public health concerns due to their complex causes and significant impact on quality of life. A key challenge in identifying these conditions is that their symptoms often overlap and are influenced by a mix of lifestyle and health-related factors. This study focuses on developing a machine learning-based system to predict sleep disorders and deploying it as an interactive full-stack web application. A dataset of 774 records was prepared by merging two synthetic datasets obtained from Kaggle, containing 11 input features related to demographic information, sleep patterns, lifestyle habits, and health indicators with sleep disorder as the target variable. During preprocessing, numerical features were standardized using StandardScaler. To handle class imbalance, a combination of SMOTE and class weights was applied to improve model learning across all classes. Several machine learning models, including Logistic Regression, Random Forest, Gradient Boosting, and XGBoost, were trained and compared using accuracy, precision, recall, and F1-score. Among the evaluated models, Random Forest achieved the highest overall accuracy, while XGBoost demonstrated stronger balanced classification performance and better handling of minority classes. Therefore, XGBoost was selected as the final predictive model for the system. Further performance improvement was achieved through hyperparameter tuning using GridSearchCV, increasing the model accuracy to 0.81. The tuned XGBoost model was then selected as the final model and integrated into the system. The application allows users to enter their health and lifestyle information through an interactive interface and receive instant predictive results. The frontend was developed using React, while the backend was implemented using Python FastAPI. In conclusion, this study demonstrates that optimized ensemble learning models can effectively support sleep disorder prediction and can be practically applied in real-world healthcare systems through web-based platforms.

Keywords: Sleep Disorders, Machine Learning, Data Preprocessing, SMOTE, XGBoost, GridSearchCV, Full-Stack Web Application, Healthcare Prediction

Table of Contents

Chapter 1 - Introduction	7
1.1 Project Overview	7
1.2 Problem Statement	8
1.3 Scope of the Project	8
Chapter 2 – Background, Objectives and Deliverables	9
2.1 Background	9
2.2 Project Objectives	9
2.3 Project Deliverables	10
2.4 Technologies Used	11
Chapter 3 - Literature Review	12
Chapter 4 – Methodology and System Design	14
4.1 Development Methodology	14
4.2 Requirements Elicitation and Survey	14
4.3 Data Collection and Preprocessing	15
4.4 Machine Learning Model Development and Evaluation	17
4.5 System Architecture	19
4.6 Database Design	20
4.7 System Features and Functional Design	21
4.8 Limitations of the Approach	22
Chapter 5 – Requirements Specification	23
5.1 Functional Requirements	23
5.2 Non-Functional Requirements	24
5.3 Hardware and Software Requirements	26
Chapter 6 – System Implementation	27
6.1 Frontend Implementation	27
6.1.1 User Authentication Interface	27
6.1.2 Routing and Navigation	28
6.1.3 Prediction Input Interface	28

6.1.4 Prediction Result Interfaces	29
6.1.5 User Dashboard.....	30
6.1.6 Prediction History Interface	31
6.1.7 Profile Management Interface.....	32
6.1.8 Admin Panel.....	32
6.1.9 Alerts and Notifications	33
6.2 Backend Implementation	34
6.2.1 API Design	34
6.2.2 Authentication and Authorization	35
6.2.3 Recommendation System (LLM Integration).....	35
6.2.4 Feature Engineering and Data Enrichment	36
6.2.5 Database Integration	37
6.2.6 System Workflow and Data Processing	37
6.3 Machine Learning Model Integration	37
6.3.1 Model Integration.....	37
6.3.2 Prediction Generation Process	38
6.4 System Testing	38
6.4.1 API Testing.....	38
6.4.2 Functional Testing.....	39
6.4.3 Input Validation Testing	40
6.4.4 Testing Scope and Limitations	40
Chapter 7 – End-Project Report.....	41
7.1 Project Summary.....	41
7.2 Evaluation of Project Objectives	41
7.3 Realization of Business Objectives.....	41
7.4 Changes made during the Project	42
Chapter 8 – Project post-mortem	42
Chapter 9 – Conclusions.....	43
References	44
Bibliography	45
Chapter 10 - Appendices	46

Appendix 10.1 – User Guide	46
Appendix 10.2 - Project Source Code.....	48
Appendix 10.3 - GitHub Repository	48
Appendix 10.4 - Project Initiation Document (PID).....	48
Appendix 10.5 - Interim.....	48
Appendix 10.6 – Record of Supervisory Meetings.....	48
Appendix 10.7 – Model Training.....	49
Appendix 10.8 – Supplementary Project Materials	49
Appendix 10.8.1 – Technology Stack mapping	49
Appendix 10.8.2 – Dataset Features	51
Appendix 10.8.3 – Survey	52
Appendix 10.8.4 – UI screenshots	55

List of Figures

Figure 4.1: Sleep Disorder Class Distribution	16
Figure 4.2: Performance Evaluation of the saved model.....	18
Figure 4.3: Analyzing Feature Importance using XGBoost.....	19
Figure 4.4: High-level Architectural Diagram	20
Figure 4.5 : Users Collection	20
Figure 4.6 : Predictions Collection	21
Figure 6.1 : Prediction Input Interface.....	28
Figure 6.2 : Sleep Apnea Results Interface	29
Figure 6.3 : User Dashboard Overview	30
Figure 6.4 : Prediction Detail Interface.....	31
Figure 6.5 : Admin Panel Dashboard Overview	33
Figure 6.6 : Postman API response	34
Figure 6.7 : Sleep Apnea personalized recommendations	35
Figure 6.8 : Insomnia personalized recommendations	36
Figure 6.9 : Model Serialization	38
Figure 10.1: Average Sleep Duration Per Night (In Hours).....	52
Figure 10.2: Frequency of Night-time Wakeups.....	53
Figure 10.3: Daytime Fatigue Frequency	53
Figure 10.4: Snoring and Breathing Disruptions During Sleep.....	54
Figure 10.5: Screen Use Before Sleep	54
Figure 10.6: Landing page UI.....	55
Figure 10.7: User Registration Form	56
Figure 10.8: User Login Form	56
Figure 10.9: Forgot Password Form	57
Figure 10.10: Insomnia Prediction Result page.....	57
Figure 10.11: Healthy Prediction Result Page.....	58
Figure 10.12: System Feedback Notifications	58
Figure 10.13: Prediction History Page.....	59
Figure 10.14: User Profile Page.....	60
Figure 10.15: Admin Panel – Registered Users Overview	61
Figure 10.16: Admin Panel – User Predictions Overview	61

Chapter 1 - Introduction

1.1 Project Overview

Over the past decade, digital technologies have significantly transformed healthcare systems by improving the way medical conditions were identified, monitored, and managed. Among the emerging approaches, machine learning had proved particularly effective in analyzing complex health-related datasets and identifying patterns that supported early detection and clinical decision-making.

In this context, the project **SleepSense: Personalized Sleep Disorder Prediction Using Advanced Machine Learning Techniques** focused on the development of a web-based application that predicted potential sleep disorders using user-provided lifestyle and health-related data. Supervised machine learning techniques were applied to classify users into three categories: healthy (no disorder), insomnia, and sleep apnea.

The system was implemented as a full-stack web application consisting of a responsive frontend, a secure backend, and an integrated machine learning model. Two types of users were supported: standard users and administrators.

From the user perspective, individuals were able to register, log in, and access a personalized dashboard. Sleep and lifestyle-related information could be inputted by users, after which predictions were generated along with personalized recommendations. In addition, previous predictions could be viewed and managed, allowing changes in sleep health to be tracked over time.

From the administrative perspective, an admin dashboard was provided, presenting an overview of system activity. This included user statistics, prediction trends, and visual analytics using charts such as pie charts, bar charts, and line graphs. These features supported effective system monitoring and data analysis.

Overall, the system was developed to provide an accessible platform for early-stage sleep disorder risk assessment. By combining machine learning with an intuitive interface, improved awareness was supported and proactive health-related decision-making was encouraged.

1.2 Problem Statement

An increase in sleep disorders was observed due to rapid changes in modern lifestyles. Factors such as academic pressure, occupational stress, irregular sleep schedules, prolonged screen exposure, and reduced physical activity had significantly disrupted natural sleep patterns. Despite this growing concern, many sleep disorders such as insomnia and sleep apnea remained undiagnosed, particularly in their early stages.

Traditional diagnostic methods, such as clinical consultations and sleep studies like polysomnography, were often expensive, time-consuming, and not easily accessible to the general population. As a result, early symptoms including fatigue, poor concentration, and irregular sleep cycles were frequently ignored or misinterpreted.

Existing digital sleep assessment tools also had limitations. Many were based on static questionnaires and lacked the ability to employ data-driven techniques for accurate and personalized predictions. This reduced their effectiveness in identifying potential risks at an early stage.

Therefore, there was a clear need for an accessible, intelligent, and data-driven system that could analyze user-specific lifestyle and health data to support early-stage prediction of sleep disorders. This project addressed this gap through the development of a machine learning-based web application for personalized sleep disorder risk assessment.

1.3 Scope of the Project

The scope of the system was limited to the early-stage prediction and awareness of sleep disorders. The project focused on predicting three sleep health categories: No Disorder (Healthy), Insomnia, and Sleep Apnea using user-provided information such as age, gender, occupation, sleep duration, height, weight, physical activity level, and optional physiological indicators including heart rate, blood pressure, and daily step count. The system also included secure user authentication, storage of prediction records with timestamps, and a user dashboard for viewing prediction history, latest prediction results, and risk-related insights. In addition, an administrative dashboard was implemented to support system monitoring and data visualization.

The project was designed as a web-based full-stack application integrated with machine learning functionality for predictive analysis. However, the system was not intended to provide clinical diagnosis, medical treatment, or professional healthcare recommendations. Integration with wearable devices and real-time health monitoring systems was not included in the project scope. Furthermore, advanced deep learning techniques requiring large-scale datasets were excluded, and the system did not include dedicated mobile applications or multilingual support beyond standard web responsiveness.

Chapter 2 – Background, Objectives and Deliverables

2.1 Background

Sleep was considered a fundamental biological process that played a critical role in maintaining physical, mental, and emotional well-being. It contributed significantly to cognitive performance, emotional regulation, immune system functioning, and overall health stability. Despite its importance, sleep health has increasingly been affected by modern lifestyle patterns, with conditions such as insomnia and sleep apnea becoming more prevalent across various population groups.

The growing availability of health-related data, combined with advances in data science and machine learning, has opened new possibilities for analyzing such conditions in a structured and scalable manner. Machine learning methods demonstrated the ability to extract meaningful patterns from complex datasets and identify relationships between lifestyle factors and health outcomes. These capabilities provided a strong technical foundation for developing predictive systems capable of supporting early awareness of sleep-related risks.

Within this context, SleepSense was developed as a data-driven web application that applied supervised machine learning to support early-stage sleep disorder risk assessment. The system was designed to make this capability accessible to general users through an intuitive interface, without requiring clinical expertise or specialist equipment.

2.2 Project Objectives

The primary objective of this project was to design and develop a web-based predictive system that supported early-stage risk assessment of sleep disorders using machine learning techniques. To achieve this aim, the following specific objectives were defined:

- To collect and preprocess sleep-related datasets from publicly available sources, addressing data quality issues such as class imbalance, to produce a reliable dataset suitable for machine learning.
- To perform exploratory data analysis to identify patterns and relationships between lifestyle factors and sleep disorder outcomes.
- To implement, train, and evaluate multiple supervised machine learning classification models, and select the most suitable algorithm based on performance metrics including accuracy, precision, recall, and macro F1-score.
- To develop a full-stack web application that enabled users to input lifestyle and health-related data and receive personalized sleep disorder risk predictions.

- To integrate the selected machine learning model into the backend system for real-time prediction processing, alongside a personalized recommendation feature powered by a large language model.
- To implement secure user authentication and role-based access control, supporting both standard users and administrators.
- To provide users with the ability to view, manage, and track their prediction history over time.
- To develop an administrative dashboard with data visualization capabilities for system monitoring and usage analysis.
- To ensure the system meets defined functional, non-functional, and performance requirements throughout development.

2.3 Project Deliverables

The following outputs were produced as part of this project:

- A functional full-stack web application, comprising a React frontend, FastAPI backend, and MongoDB Atlas database, supporting sleep disorder risk prediction, personalized recommendations, and prediction history management for registered users, alongside a dedicated administrative monitoring interface.
- A trained and serialized XGBoost classification model, stored in .pkl format, capable of predicting three sleep health categories from structured user input data.
- A RESTful backend system handling authentication, prediction processing, database operations, and integration with the Google Gemini API for personalized recommendation generation.
- A responsive frontend application providing an accessible and user-friendly interface across all system functionalities, including user authentication, prediction input, history management, and profile control.
- An administrative dashboard providing system-level monitoring capabilities, including visualization of user statistics, prediction outcome distributions, and usage trends through interactive charts.

- Comprehensive project documentation comprising the interim report, final report, and a user guide detailing system installation and operation.
- A structured source code repository maintained throughout development to ensure transparency, maintainability, and reproducibility. The complete source code is provided in Appendix 10.2, and the GitHub repository along with commit history is provided in Appendix 10.3.

2.4 Technologies Used

The system was developed using a combination of frontend, backend, machine learning, and database technologies. React.js was selected for the frontend due to its component-based architecture, which supported modular and responsive development. FastAPI was adopted as the backend framework for its high performance, asynchronous request handling, and built-in data validation capabilities. Python was used as the primary backend language, given its extensive support for machine learning libraries and data processing tools.

The machine learning pipeline was built using Scikit-learn for preprocessing and evaluation, and XGBoost as the classification library. Imbalanced-learn was employed to apply SMOTE during training to address minority class representation. MongoDB Atlas was chosen as the database for its flexible document-based schema and cloud-based scalability. Secure authentication was implemented using JSON Web Tokens (JWT) to protect API endpoints and manage user sessions. The Google Gemini API was integrated to generate personalized, context-aware sleep recommendations based on prediction outcomes. A full mapping of all system components and their corresponding technologies including supporting libraries and development tools is provided in Appendix 10.8.1.

Chapter 3 - Literature Review

The application of machine learning techniques to the detection and classification of sleep disorders such as insomnia and sleep apnea had gained significant attention in recent years, driven by advances in artificial intelligence and the growing availability of health-related datasets. For this project, relevant literature was identified through academic databases including Google Scholar, IEEE Xplore, and ScienceDirect. Search terms employed included "sleep disorder prediction," "machine learning in sleep health," "AI-based sleep disorder detection," and "sleep apnea classification." The selected studies were reviewed to understand commonly used algorithms, datasets, evaluation metrics, and practical limitations, with particular attention to identifying gaps that informed and justified the development of SleepSense.

Several studies investigated the application of traditional machine learning models to sleep disorder prediction using physiological and lifestyle parameters. A comparative study evaluated multiple classification algorithms including Logistic Regression, Decision Trees, Random Forest, Support Vector Machines, Gradient Boosting, and Neural Networks. Ensemble methods, particularly Random Forest and XGBoost, achieved prediction accuracy of approximately 93%, demonstrating strong predictive performance. However, the system was developed primarily for clinical use and was not designed to be accessible to general users (Airlangga, 2024).

Similarly, a further study examined the combination of machine learning algorithms with wearable device data for real-time sleep disorder detection. Decision Trees achieved the highest accuracy of 93.5% among the models evaluated, which included Logistic Regression and XGBoost. Whilst the study highlighted the potential of AI-assisted sleep management, practical deployment constraints were identified as a significant barrier to real-world adoption (Yadav et al., 2023).

In a third study, a Random Forest classifier was applied to the Sleep Health and Lifestyle Dataset following preprocessing steps including encoding, normalization, and SMOTE-based class balancing. The model achieved an accuracy of 96.70% with strong precision, recall, and F1-scores across the Healthy, Insomnia, and Sleep Apnea classes, demonstrating that non-invasive lifestyle and physiological data could be used effectively for sleep disorder classification. However, the study did not address end-user accessibility or the practical deployment of the solution as a usable application (Ravikumar et al., no date).

Recent literature placed increasing emphasis on ensemble and advanced machine learning methods to improve prediction reliability, particularly for imbalanced sleep datasets. One study introduced a multi-model approach incorporating voting classifiers, stacking, and SMOTE-based data balancing. Accuracies of 96.88% were achieved on SMOTE-balanced datasets and above 95% on original datasets, further validating the suitability of ensemble learning for sleep quality prediction tasks (Mostafa Monowar *et al.*, 2024).

A further study explored Artificial Neural Network ensemble approaches, including ANN Bagging, Boosting, and Weighted Ensemble configurations, achieving accuracies of 94.7% and 92.9% respectively. Despite the technical sophistication of the approach, the work remained confined to an academic context without any practical user-facing application (Alom *et al.*, 2025).

Additionally, a study employing Gradient Boosting, AdaBoost, and LSTM networks as non-invasive alternatives to polysomnography-based diagnosis reported the highest accuracy of 96% with an F1-score of 95.1% using Gradient Boosting. The study reinforced the potential of machine learning for cost-effective sleep disorder detection, though the focus remained on algorithmic performance rather than system usability (Sarala *et al.*, no date).

Class imbalance represented a recurring challenge across sleep disorder datasets, where instances of the healthy class significantly outnumbered those representing disorder conditions. One study addressed this limitation through the application of SMOTE and ADASYN data balancing techniques, evaluating classifiers including Logistic Regression, SVM, Random Forest, XGBoost, and a stacking ensemble. The stacking model achieved the highest accuracy of 94%, demonstrating the effectiveness of ensemble learning in improving prediction reliability on imbalanced data (Kumar *et al.*, no date).

A further limitation identified across the reviewed literature was the absence of interpretability and user-oriented presentation of results. Although models such as Random Forest and Gradient Boosting offered variable importance metrics, these outputs were rarely translated into meaningful, accessible feedback for non-technical users. Additionally, most existing solutions were confined to laboratory or academic settings and lacked the practical infrastructure required for web-based deployment.

Whilst considerable progress had been made in machine learning-based sleep disorder prediction, a clear gap existed in the literature. The predominant focus across reviewed studies was on maximizing predictive accuracy, with limited attention given to accessibility, interpretability, and real-world usability. Very few studies addressed the development of systems that allowed users to submit basic input parameters and receive understandable screening outcomes through a web-based interface in real time. SleepSense was developed to address these identified gaps by combining supervised machine learning with a full-stack web application, integrating a tuned XGBoost classifier with a FastAPI backend and a React-based frontend. Unlike the studies reviewed, which were predominantly concerned with model optimization, this project prioritized the translation of predictive capability into a practical, user-accessible tool, bridging the gap between academic research and real-world applicability in sleep health awareness.

Chapter 4 – Methodology and System Design

4.1 Development Methodology

An iterative and data-driven development approach was adopted for this project by combining principles of Agile methodology with a structured machine learning workflow. Instead of following a rigid linear process, development was carried out progressively, allowing each stage to inform and refine subsequent stages.

The development process was organized into four main phases: data collection and preprocessing, machine learning model development and evaluation, backend implementation, and frontend development. Within each phase, iterative cycles were followed in which requirements were reviewed, outputs were evaluated, and improvements were incorporated based on the results of the preceding stage. Insights obtained during exploratory data analysis influenced preprocessing decisions, while model evaluation results guided the feature engineering techniques applied before final model training.

This flexible approach enabled issues to be identified and resolved incrementally throughout development rather than only at the final stage. As a result, continuous improvements were achieved across both the machine learning pipeline and the web application components during the project lifecycle.

4.2 Requirements Elicitation and Survey

To validate the relevance of the problem domain and support the requirements gathering process, a structured online survey was conducted prior to system development. The survey was distributed via Google Forms across online channels and received 120 responses from participants spanning a range of ages and occupations. It is important to note that the survey data was not used in the training or evaluation of the machine learning model; its purpose was solely to confirm that sleep-related concerns were genuinely present within the general population.

The findings provided clear contextual evidence supporting the project's problem statement. Regarding sleep duration, 24.2% of respondents reported sleeping fewer than 6 hours per night, falling below the recommended 7 to 9 hours, with many respondents sleeping between 6 and 8 hours. Sleep continuity was also a concern, with 66.7% of participants reporting at least one night-time wakeup, of whom 39.2% woke once and 25.8% woke two to three times per night. Breathing-related disruptions were reported by 64.2% of respondents at least occasionally, indicating the presence of potential sleep apnea-related symptoms within the surveyed population. Daytime fatigue was experienced by many participants at least occasionally, further reflecting the impact of disrupted sleep on daily functioning. Screen exposure before sleep was also prevalent, with 94.2% of respondents reporting use of a phone, tablet, or television immediately prior to sleeping.

These findings collectively confirmed that sleep disruption was a common and largely unaddressed concern across the general population, supporting the case for an accessible, data-driven sleep disorder risk assessment tool. The critical survey questions and key response distributions are provided in Appendix 10.8.3.

4.3 Data Collection and Preprocessing

The data collection used in this project was obtained from publicly available Kaggle repositories and comprised sleep health-related data integrating both physiological and lifestyle-related attributes. To improve dataset diversity and reduce potential bias arising from a single data source, two compatible synthetic datasets were merged. The final dataset consisted of 774 records, capturing key indicators such as sleep duration, physical activity level, stress level, and physiological health metrics. A full description of all dataset features, including data types and value ranges, is provided in this Appendix 10.8.2.

A comprehensive preprocessing pipeline was applied to ensure the quality and suitability of the dataset for machine learning tasks. The dataset was initially examined for missing values, duplicate records, and formatting inconsistencies. During this process, 509 missing values were identified in the target variable, which corresponded to individuals without diagnosed sleep disorders. These missing entries were therefore imputed with the “None” class label to preserve dataset completeness and maintain the natural distribution of the healthy population, avoiding unnecessary class imbalance.

In addition, duplicate records and structural inconsistencies were checked and addressed to ensure data integrity. Outlier analysis was also performed on numerical features such as age and sleep-related physiological indicators. Although several extreme values were detected, these observations were retained, as the system is intended for real-world deployment where natural variability across individuals is expected. Retaining these values improves the model’s ability to generalize to diverse user inputs, including edge cases such as elderly individuals or users with atypical sleep patterns. This approach ensured that the dataset remained both realistic and representative of real-world conditions while maintaining suitability for machine learning model development.

Feature engineering was then carried out to improve the representation of health-related variables. Specifically, the blood pressure attribute was separated into its systolic and diastolic components. This transformation was justified on clinical grounds, as the two measures captured distinct aspects of cardiovascular function and allowed the model to detect cardiovascular risk patterns more precisely.

Categorical variables, including gender and occupation, were encoded using one-hot encoding to avoid imposing any ordinal relationship on nominal data. The BMI category was encoded using ordinal encoding, given its inherently ordered structure, so that meaningful distinctions between weight categories were retained. The target variable, sleep disorder class, was label encoded to support multi-class classification.

Numerical variables were standardized to bring all features to a comparable scale. This step was particularly important for distance-based and gradient-sensitive algorithms, where differences in feature magnitude could disproportionately influence model outputs.

The dataset was then split into training and testing sets using an 80:20 ratio. To address class imbalance present in the training data, the Synthetic Minority Oversampling Technique (SMOTE) was applied exclusively to the training set. Restricting SMOTE to the training partition ensured that synthetic samples did not influence the evaluation process, thereby preventing data leakage.

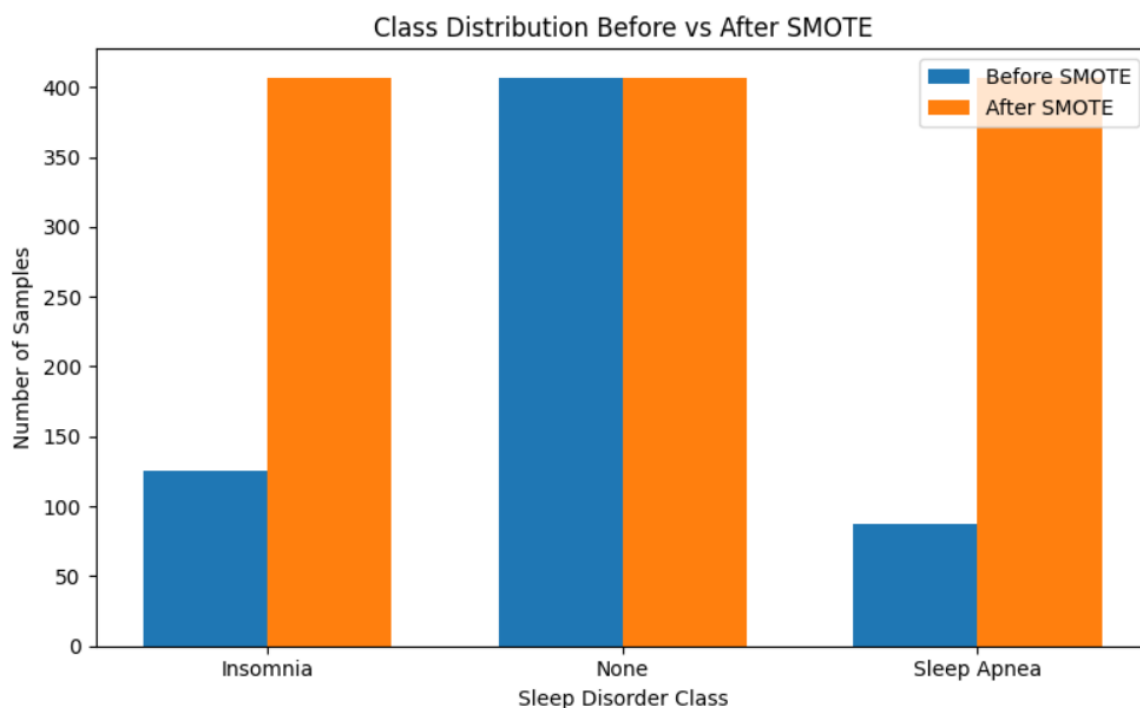


Figure 4.1: Sleep Disorder Class Distribution

4.4 Machine Learning Model Development and Evaluation

Multiple supervised machine learning models were developed and evaluated using the preprocessed dataset. The primary aim was to identify a model capable of accurately classifying sleep disorder categories whilst remaining robust under class imbalance conditions. Four classification algorithms were selected for comparison: Logistic Regression, Random Forest, Gradient Boosting, and XGBoost. These were chosen because their underlying learning mechanisms differed substantially, allowing evaluation across both linear and non-linear decision boundaries. All models were trained and assessed on the same preprocessed dataset under identical experimental conditions to ensure a fair comparison.

Performance was evaluated using accuracy, precision, recall, and macro F1-score. The macro F1-score was treated as the primary evaluation metric, since accuracy alone can be misleading when class distributions are uneven. By computing the F1-score independently for each class and averaging the results, this metric provided a more balanced assessment of model behavior across all three sleep disorder categories. To further address class imbalance during training, class weighting was applied where supported, assigning greater importance to the minority classes, Insomnia and Sleep Apnea, without modifying the original data distribution.

Among the evaluated models, Random Forest achieved the highest overall accuracy. However, XGBoost demonstrated slightly stronger balanced classification performance across minority classes and produced the highest macro F1-score overall. Since the project involved deployment within a real-time web-based prediction system, greater emphasis was placed on balanced predictive capability rather than accuracy alone. Consequently, XGBoost was selected as the final predictive model due to its stronger generalization performance and improved handling of underrepresented classes.

Hyperparameter tuning was subsequently conducted on the selected model using five-fold cross-validation. The decision to tune only the selected model was deliberate, as concentrating computational effort on the most suitable candidate was considered more effective than uniformly tuning all four models. The cross-validation strategy ensured that the selected parameters remained stable across different data splits rather than becoming overly optimized to a single partition.

The final tuned XGBoost model achieved an accuracy of approximately 0.81 and a macro F1-score of 0.72. The model was saved and integrated into the backend of the web application, enabling real-time sleep disorder prediction. Although this accuracy is marginally lower than figures reported in certain related studies, this can largely be attributed to the increased heterogeneity introduced through dataset merging. Whilst this additional variability increased the complexity of the learning task, it also improved the model's applicability to more realistic and diverse input scenarios.

Computational efficiency was also evaluated as part of the assessment process. The serialized model occupied approximately 1.5 MB of storage and achieved an average inference time of 0.007 seconds, confirming its suitability for deployment in resource-constrained and real-time environments. Analyzing feature importance was carried out to identify the variables contributing most significantly to prediction outcomes, supporting the interpretability of the model, a particularly important consideration in health-related predictive systems.

Table 4-1 Comparison of Machine Learning Model performance

Model	Accuracy	Macro F1-Score	Precision (Macro Avg)	Recall (Macro Avg)
Random Forest	0.7871	0.6913	0.79	0.64
Gradient Boosting	0.7677	0.6798	0.71	0.66
Logistic Regression	0.6903	0.6228	0.62	0.63
XGBoost	0.7806	0.6931	0.74	0.67

FINAL MODEL (Tuned XGBoost)

Accuracy: 0.8064516129032258

Macro F1: 0.7200072621641249

	precision	recall	f1-score	support
0	0.66	0.61	0.63	31
1	0.83	0.93	0.88	102
2	0.92	0.50	0.65	22
accuracy			0.81	155
macro avg	0.80	0.68	0.72	155
weighted avg	0.81	0.81	0.80	155

Figure 4.2: Performance Evaluation of the saved model

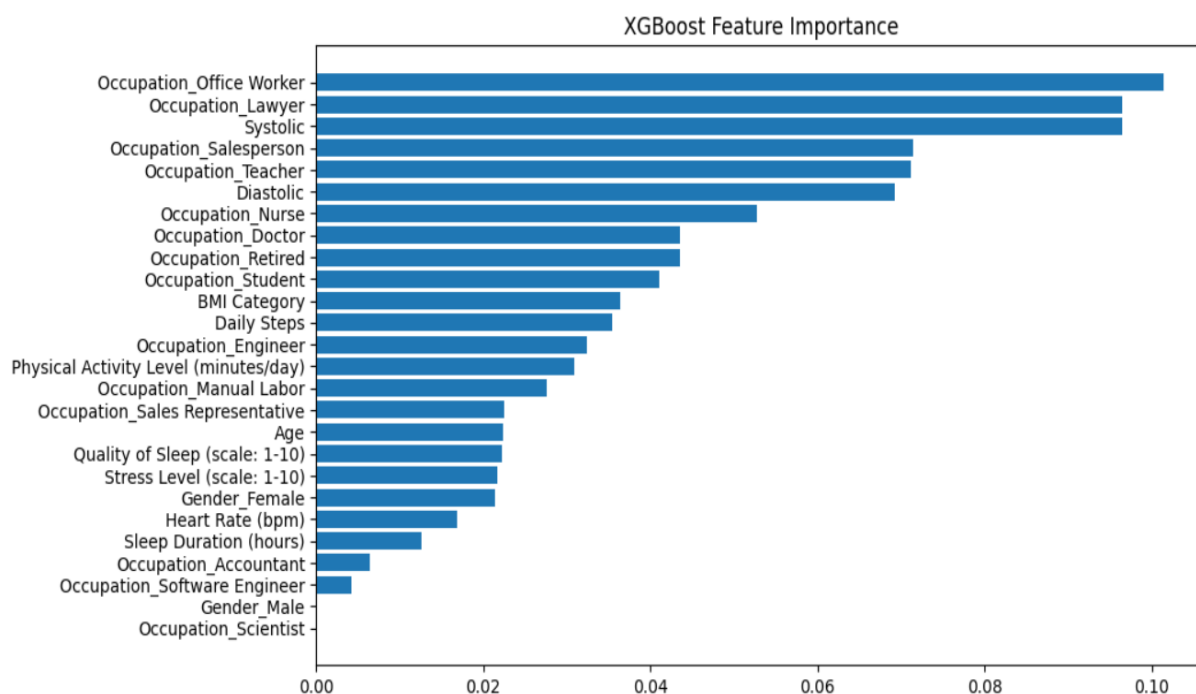


Figure 4.3: Analyzing Feature Importance using XGBoost

4.5 System Architecture

The SleepSense system was designed as a full-stack web application structured around three core components: a React-based frontend, a FastAPI backend, and an integrated machine learning model, supported by a MongoDB Atlas cloud database. A modular architecture was adopted to ensure separation of concerns, maintainability, and scalability across all layers of the system. Each component communicated through RESTful API requests, with the backend serving as the central processing layer coordinating data flow between the frontend, the database, and the prediction model.

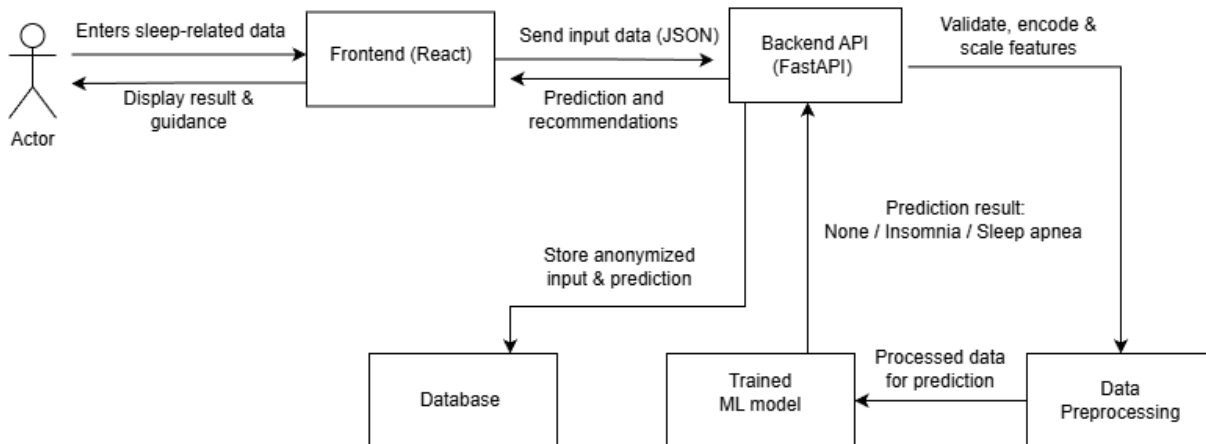


Figure 4.4: High-level Architectural Diagram

4.6 Database Design

MongoDB Atlas was used as a cloud-based NoSQL database for storing and managing application data. The database, named *sleepdb*, consisted of two primary collections.

The Users collection stored profile information for each registered account, including name, email, assigned role, and account creation timestamp. This collection supported authentication and role-based access control, differentiating between standard users and administrators.

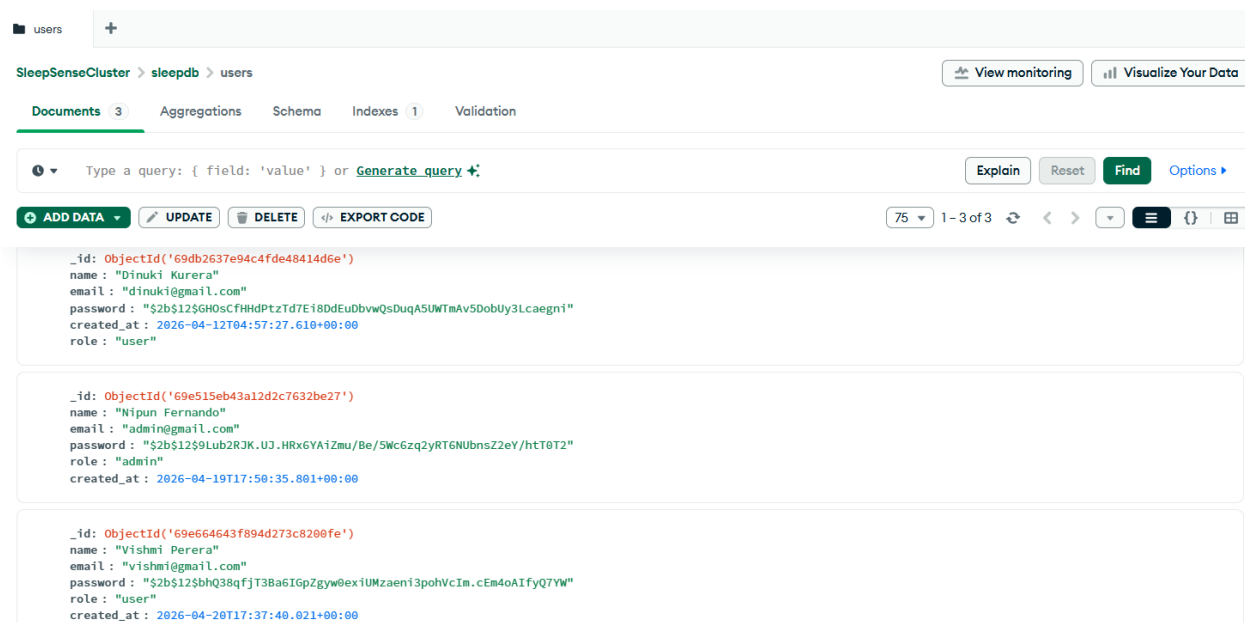


Figure 4.5 : Users Collection

The Predictions collection stored all prediction records generated by the system. Each record included the user-submitted input data, the predicted sleep disorder class, the associated risk level, and a timestamp. Records were linked to user accounts through a unique user identifier, ensuring that each user could only access their own prediction history.

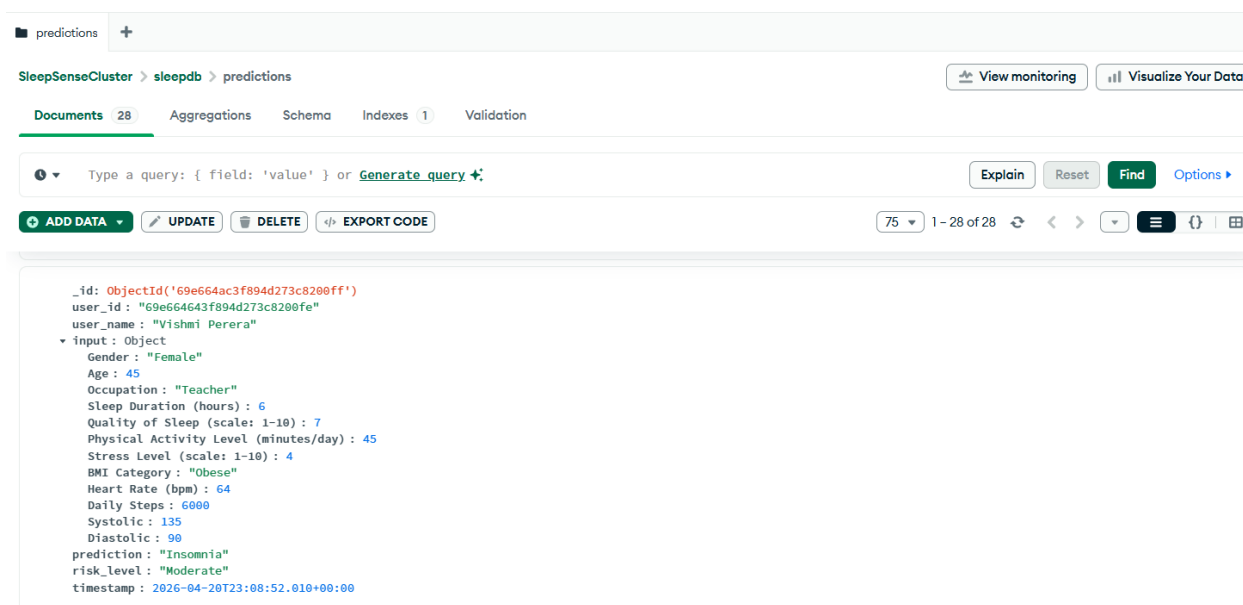


Figure 4.6: Predictions Collection

4.7 System Features and Functional Design

The system was designed around two distinct user roles, standard users and administrators, each with a dedicated set of features and interface views. This role-based structure ensured that system functionality was appropriately scoped and that sensitive administrative data remained inaccessible to general users.

For standard users, the core interaction flow was centered on the prediction process: inputting lifestyle and health-related data, receiving a sleep disorder risk classification, and reviewing personalized recommendations. Supporting features including prediction history management, profile management, and password reset functionality were included to encourage continued engagement and account control.

The administrative interface was designed separately to provide system-level oversight, including user management and visualization of prediction trends and usage statistics. This separation ensured that monitoring capabilities did not interfere with the standard user experience.

4.8 Limitations of the Approach

Whilst the system delivered meaningful predictive insights, several limitations were acknowledged. The model was trained exclusively on publicly available and synthetic datasets, which may not have fully captured the variability present in real-world clinical populations. The use of merged synthetic data, whilst beneficial for improving dataset diversity, meant that the model had not been validated against clinically verified sleep disorder diagnoses.

The system was designed for early-stage risk awareness only and was not intended to serve as a substitute for clinical diagnosis or medical treatment. Users were encouraged to seek professional healthcare evaluation for any concerns identified through the system.

From a technical standpoint, all development and testing were conducted within a local environment under single-user conditions. The system was not deployed to a live server during the project period, and external user testing was therefore not carried out.

Additionally, the prediction model relied on structured user-provided input. No integration with wearable devices or real-time physiological monitoring was included within the project scope, which limited the system's ability to capture dynamic health data.

Chapter 5 – Requirements Specification

5.1 Functional Requirements

This section defines the functional requirements of the system, specifying the capabilities that the system was required to provide for both standard users and administrators.

User Authentication and Account Management

- Users were required to register by providing the necessary personal information to create an account.
- Registered users were authenticated securely through a login mechanism to ensure controlled access to the system.
- Access to prediction features and personal data was restricted to authenticated users only.
- A password reset facility was required to allow users to recover account access securely when needed.
- Users were required to be able to log out and to delete their accounts, with all associated data removed upon deletion.

Prediction Functionality

- Users were required to input relevant lifestyle and sleep-related information, including gender, age, occupation, sleep duration, height, weight, and physical activity level.
- Users were permitted to optionally provide additional physiological data, including heart rate, daily steps, systolic blood pressure, and diastolic blood pressure.
- Required model input features not directly provided by the user were automatically derived to ensure complete prediction inputs.
- All user inputs were validated prior to processing to ensure correctness, completeness, and acceptable value ranges, with appropriate error messages displayed for invalid entries.
- Prediction results were required to indicate whether the user was likely to have no sleep disorder, insomnia, or sleep apnea, presented in a clear format suitable for non-technical users.
- Personalized sleep-related recommendations were required to be generated based on prediction outcomes to support user awareness and lifestyle improvement.

Prediction History and Profile Management

- Each prediction record was required to be stored securely in the database with a timestamp for future reference.
- Users were required to be able to view, filter, and delete their prediction history records.
- Users were required to be able to view their profile information and a summary of account activity.

Administrative Functions

- Administrators were required to authenticate securely using authorized credentials through a dedicated login process.
- An administrative dashboard was required to present system-level statistics, including total users, prediction outcome distributions, and usage trends.
- Administrators were required to be able to manage user accounts, including the ability to remove users from the system when necessary.

5.2 Non-Functional Requirements

The following quality-related requirements were defined to ensure the system delivered a reliable, secure, and maintainable service.

Security

- User passwords were stored using hashing techniques to prevent unauthorized access to credentials.
- Token-based authentication was implemented to protect all API endpoints, ensuring that only authenticated users could access restricted system features.
- Access to personal data and prediction records was always restricted to the respective account holder.
- All user inputs were validated and sanitized to reduce exposure to security threats such as injection attacks and malicious data entry.

Performance

- Prediction results were required to be generated within approximately three seconds of submission to ensure responsive user experience.
- The trained machine learning model was loaded into memory at server startup and reused across requests to avoid repeated loading overhead.
- The system was required to handle concurrent user requests without noticeable degradation in response time.

Usability

- The user interface was required to be clean, intuitive, and navigable by users without a technical background.
- Prediction results were required to be displayed in plain, understandable language, avoiding technical terminology where possible.
- The system was required to be accessible through modern web browsers without requiring additional software installation.

Reliability

- The system was required to maintain stable operation during runtime, particularly during model inference and database operations.
- Errors and unexpected conditions were required to be handled gracefully, preventing application crashes and preserving data integrity.
- Data stored in the database was required to remain consistent and accurately retrievable across all user sessions.

Maintainability

- The system was required to follow a modular design approach, separating concerns across frontend components, backend routes, model logic, and database operations to support future maintenance and enhancement.

5.3 Hardware and Software Requirements

Hardware Requirements

The development and testing of the system were conducted on a standard personal computer with the following minimum configuration:

- A minimum of 8 GB of RAM to support data processing, model training, and simultaneous execution of frontend and backend environments.
- A multi-core processor to handle parallel operations during development and testing.
- A minimum of 10 GB of available storage for the dataset, project dependencies, and application files.
- A stable internet connection for accessing cloud services, including MongoDB Atlas and Google Gemini API.

Software Requirements

The following software environment was required for development, testing, and execution of the system:

Runtime environment:

- Windows, Linux, or macOS operating system.
- Node.js and npm for executing the frontend application.
- Python 3.12 for running the backend server and machine learning components.
- pip for managing Python package dependencies.

Browser requirements:

- A modern web browser such as Google Chrome or Microsoft Edge for accessing the application interface.

Chapter 6 – System Implementation

6.1 Frontend Implementation

The frontend of the SleepSense system was developed using React.js, adopting component-based architecture and a single-page application (SPA) approach. This design enabled dynamic content updates without requiring full page reloads, resulting in a smooth and responsive experience for the end user. The interface was built with responsiveness as a primary concern, ensuring consistent usability across desktops, tablets, and mobile devices.

State and lifecycle management were handled through functional components and React hooks, allowing dynamic data and user interactions to be managed effectively throughout the application. Communication with the backend was established through Axios, which handled all HTTP requests and enabled real-time data exchange between the frontend and the backend for operations such as prediction generation, history retrieval, and user authentication.

The application was structured around reusable components, with a clear separation of concerns between UI elements, page-level components, and service logic. This modular approach was adopted to improve maintainability, support scalability, and simplify future enhancements. The frontend comprised several key modules: the landing page, authentication interfaces, user dashboard, prediction interface, prediction history, profile management, and the administrator panel. Each module was designed to deliver a clear and intuitive experience whilst fulfilling its specific role within the overall system.

6.1.1 User Authentication Interface

The user authentication module provided secure access to the system through dedicated login and registration interfaces. New users were able to create accounts by submitting the required personal information, whilst returning users could authenticate using their registered credentials.

Client-side validation was implemented to ensure that all inputs conformed to the required formats before being transmitted to the backend. Validation logic covered fields such as email format and password length requirements, with descriptive error messages displayed to guide users in correcting invalid entries. This reduced the volume of erroneous submissions and improved overall usability.

The authentication interface was integrated with the backend to handle credential verification and session management, with token-based session handling discussed in detail under the backend implementation section. A password reset feature was also incorporated, allowing users to securely recover account access when required.

6.1.2 Routing and Navigation

Client-side routing and navigation were managed using React Router DOM, enabling seamless transitions between pages including the login interface, dashboard, prediction interface, history page, profile section, and administrator panel without triggering full page reloads.

The routing structure was configured to support both public and protected routes. Route protection was enforced by evaluating the presence and validity of the stored JWT prior to rendering any restricted page; unauthenticated users were automatically redirected to the login interface. This mechanism ensured that sensitive areas of the application remained inaccessible to unauthorized users.

6.1.3 Prediction Input Interface

The prediction input interface allowed users to enter the lifestyle and sleep-related information required for generating a prediction. Input fields captured attributes including age, gender, occupation, sleep duration, physical activity level, height, and weight.

Form handling was implemented to capture and manage user inputs effectively throughout the submission process. Validation mechanisms ensured that all required fields were completed and that entered values fell within acceptable ranges before submission. Upon successful validation, the input data was transmitted to the backend through Axios for processing by the machine learning model. The interface was designed to guide users through the input process in a logical sequence, minimizing the likelihood of entry errors. Once submitted, the request was processed by the backend, and the resulting prediction was returned and displayed on a dedicated result page.

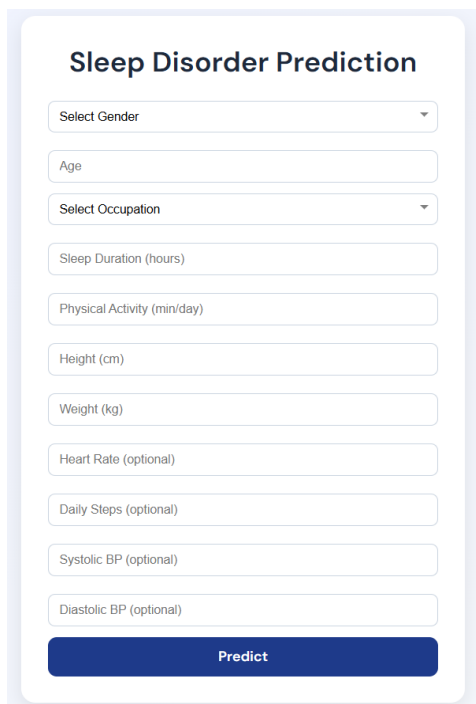
The image shows a web form titled "Sleep Disorder Prediction". It contains several input fields: a dropdown menu for "Select Gender", a text field for "Age", a dropdown menu for "Select Occupation", and text fields for "Sleep Duration (hours)", "Physical Activity (min/day)", "Height (cm)", "Weight (kg)", "Heart Rate (optional)", "Daily Steps (optional)", "Systolic BP (optional)", and "Diastolic BP (optional)". At the bottom of the form is a dark blue button labeled "Predict".

Figure 6.1: Prediction Input Interface

6.1.4 Prediction Result Interfaces

Upon successful form submission, the system generated a prediction result and directed the user to one of three condition-specific result pages; Healthy, Insomnia, or Sleep Apnea. Color-coded status indicators were applied to the result display to improve readability and support user understanding of the predicted condition. Each result page presented the predicted condition along with an overview, a relevant YouTube video, and a brief explanation of the result. Health insights associated with the predicted condition were also displayed, along with personalized recommendations generated by the system, which are discussed in detail under the backend implementation section. The figure below illustrates the Sleep Apnea result page as a representative example; the remaining result pages for Healthy and Insomnia conditions are provided in Appendix 10.8.4.

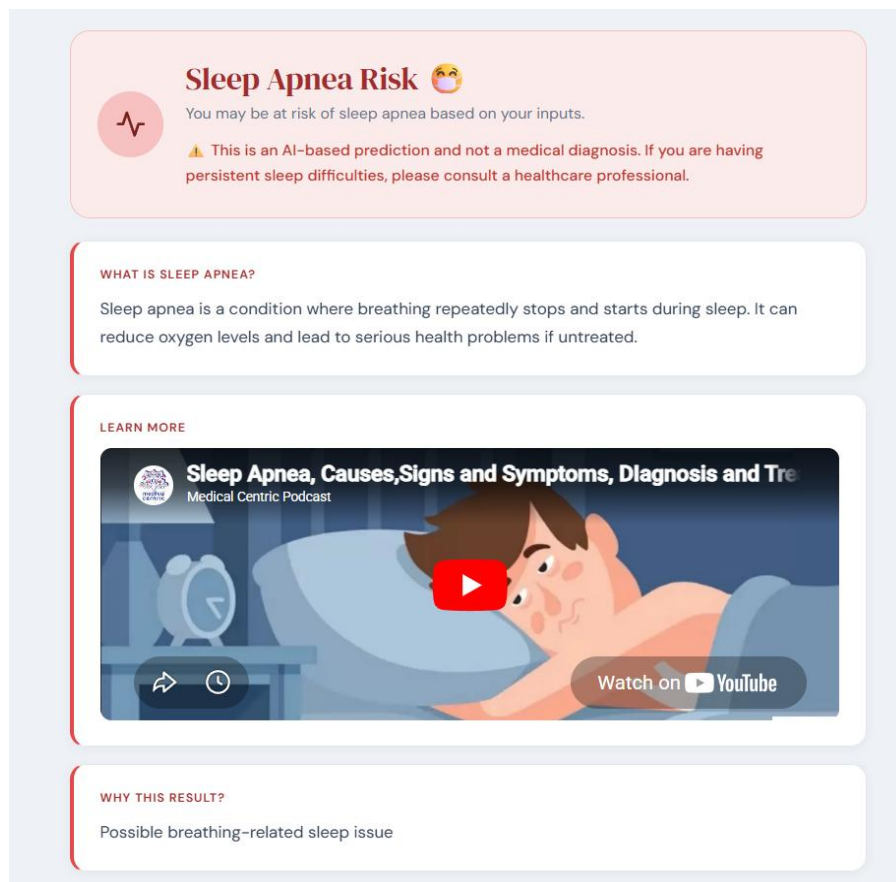


Figure 6.2: Sleep Apnea Results Interface

6.1.5 User Dashboard

The user dashboard served as the central interface of the system, providing an at-a-glance overview of user activity and prediction-related insights. Key information presented included the total number of predictions made, the most recent prediction result, and the associated risk level. These metrics were displayed using summary cards, laid out to allow users to interpret their data quickly and without ambiguity.

Core system functionalities including the initiation of new predictions and the review of recent activity were accessible directly from the dashboard. Educational content related to sleep health, including embedded video resources, was also integrated to raise user awareness and encourage healthier lifestyle practices.

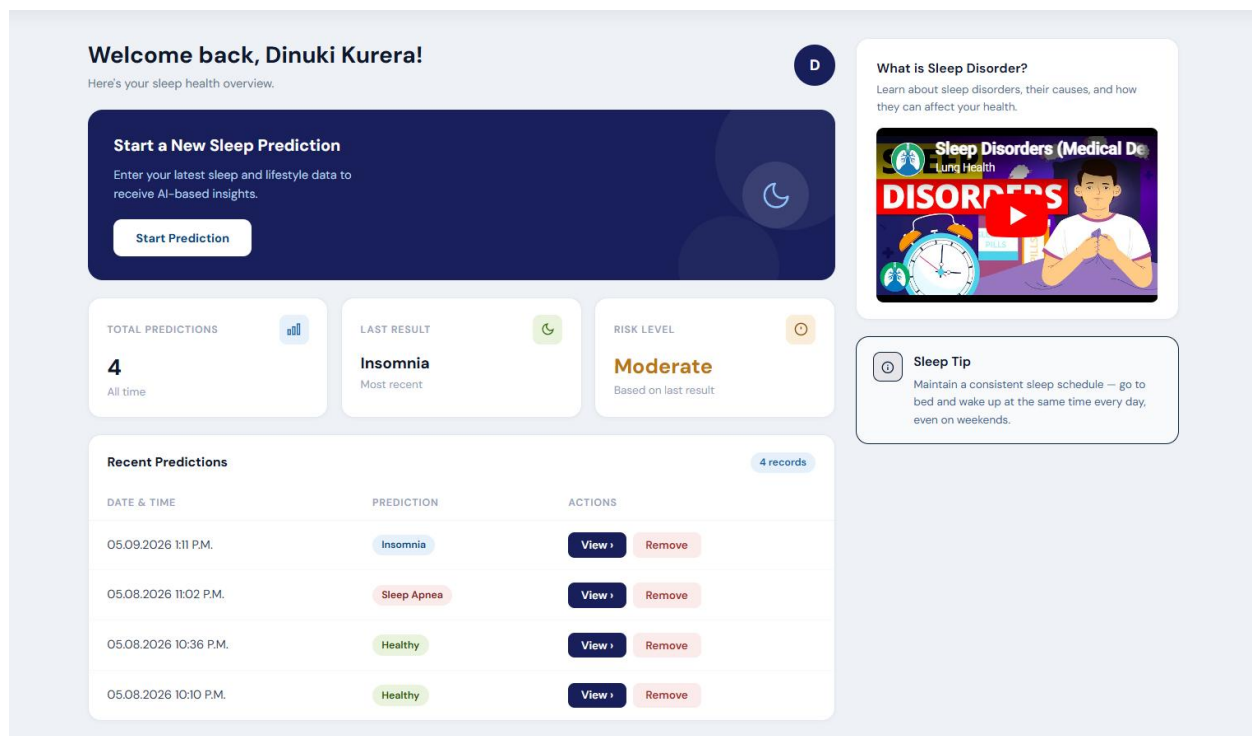


Figure 6.3: User Dashboard Overview

The dashboard also maintained a record of the user's recent predictions; each displayed with options to view or delete. Selecting the view option navigated the user to a prediction detail page, which presented the exact inputs submitted during that prediction session. This allowed users to revisit and reflect on their previously entered data at any time.

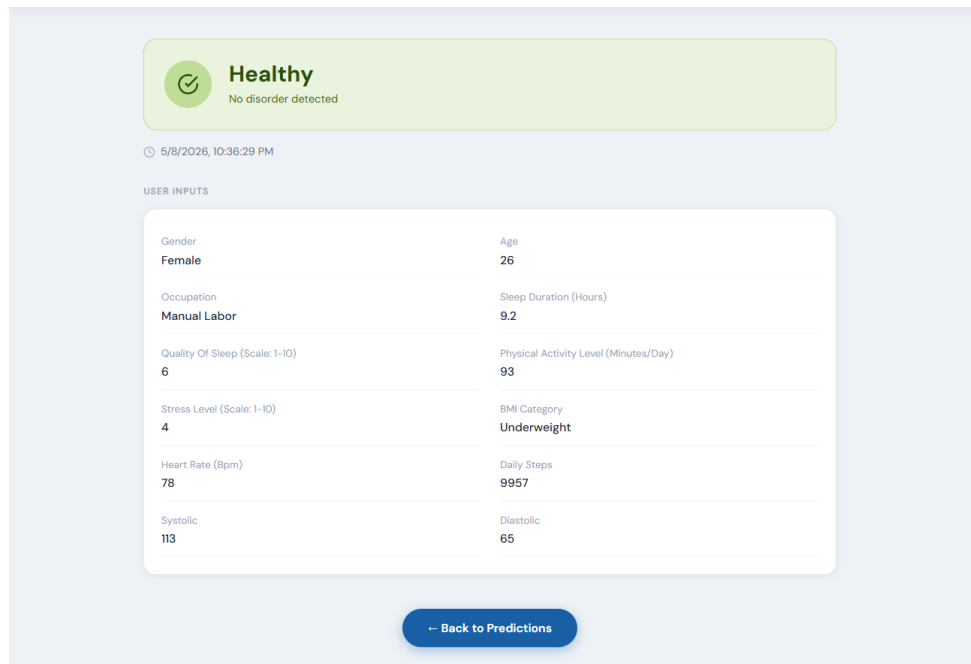


Figure 6.4: Prediction Detail Interface

6.1.6 Prediction History Interface

The prediction history interface enabled users to review all previously generated predictions in a structured and readable format. Each record displayed the relevant input data alongside the corresponding prediction outcome, giving users a complete view of each submission.

Client-side filtering functionality was implemented, allowing users to categories prediction records by outcome class specifically Healthy, Insomnia, and Sleep Apnea. This was achieved by maintaining active filter selection in component state and conditionally rendering the relevant records, enabling rapid identification of patterns across the prediction history without requiring additional server requests.

Visual summaries of historical data were also incorporated using Recharts, presenting outcome distributions and trends through interactive charts. These visualizations helped users identify changes in their sleep-related conditions over time.

6.1.7 Profile Management Interface

The profile management interface provided users with a dedicated space to view and manage their account-related information. Details presented included the registered email address, account creation date, and a summary of system activity associated with the account.

Users were able to log out securely and to delete their accounts when required. Account deletion triggered the permanent removal of all associated data from the database, ensuring that users retained full control over their personal information. The interface was designed to prioritize simplicity and accessibility whilst upholding data privacy standards.

6.1.8 Admin Panel

The administrator panel provided a centralized interface for monitoring system activity and managing user-related data. The admin dashboard displayed key system statistics, including the total number of registered users and total predictions generated within the system. To support data analysis and improve usability, several interactive visualizations were implemented using Recharts. These included a bar chart for risk level analysis representing the distribution of low, moderate, and high-risk predictions, a pie chart illustrating the distribution of predicted sleep disorder categories such as healthy, insomnia, and sleep apnea cases, and a line graph showing prediction activity over time based on recorded dates and times.

In addition to the dashboard, the administrator interface included dedicated pages for user management and prediction monitoring. The users page allowed administrators to manage registered accounts, including the ability to remove users from the system when necessary. The predictions page enabled administrators to review prediction records generated by all users, supporting effective monitoring of overall system usage. Access to the administrator panel was restricted to authorized administrator accounts to ensure the security and confidentiality of system data.

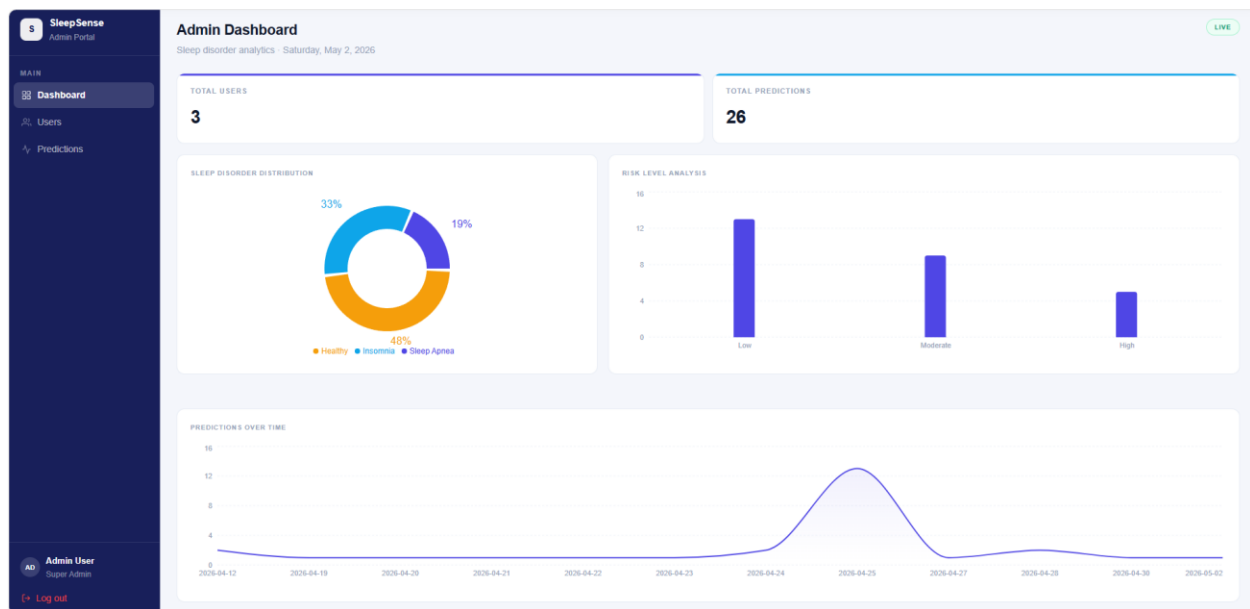


Figure 6.5: Admin Panel Dashboard Overview

6.1.9 Alerts and Notifications

An alert and notification system was incorporated throughout the application to provide users with immediate feedback following key interactions. SweetAlert2 was used to deliver modal alerts for significant events such as successful registration, account deletion confirmation, and error conditions. For lighter, non-blocking feedback, inline notifications were displayed for actions such as form validation failures and minor system messages.

Error notifications were designed to communicate the nature of the issue clearly, enabling users to identify and resolve problems without confusion. Success messages confirmed the completion of key operations, reinforcing user confidence in the system's responsiveness. The user interface screenshots for all modules presented in this section are compiled and referenced in Appendix 10.8.4 for further reference.

6.2 Backend Implementation

The backend of the system was developed using FastAPI, serving as the central processing layer responsible for handling all system operations. It functioned as the intermediary between the frontend interface, the database, and the machine learning components. A modular architecture was adopted to maintain scalability, support long-term maintainability, and simplify integration with external services.

6.2.1 API Design

A set of RESTful APIs was implemented to support core system functionalities, including user authentication, prediction requests, retrieval of prediction history, and administrative operations. Each endpoint was designed to accept structured JSON input, carry out validation, and return standardized responses to the frontend.

FastAPI's built-in validation mechanisms were used to confirm that incoming data conformed to expected formats prior to processing. This reduced the likelihood of runtime errors and contributed to overall system reliability.

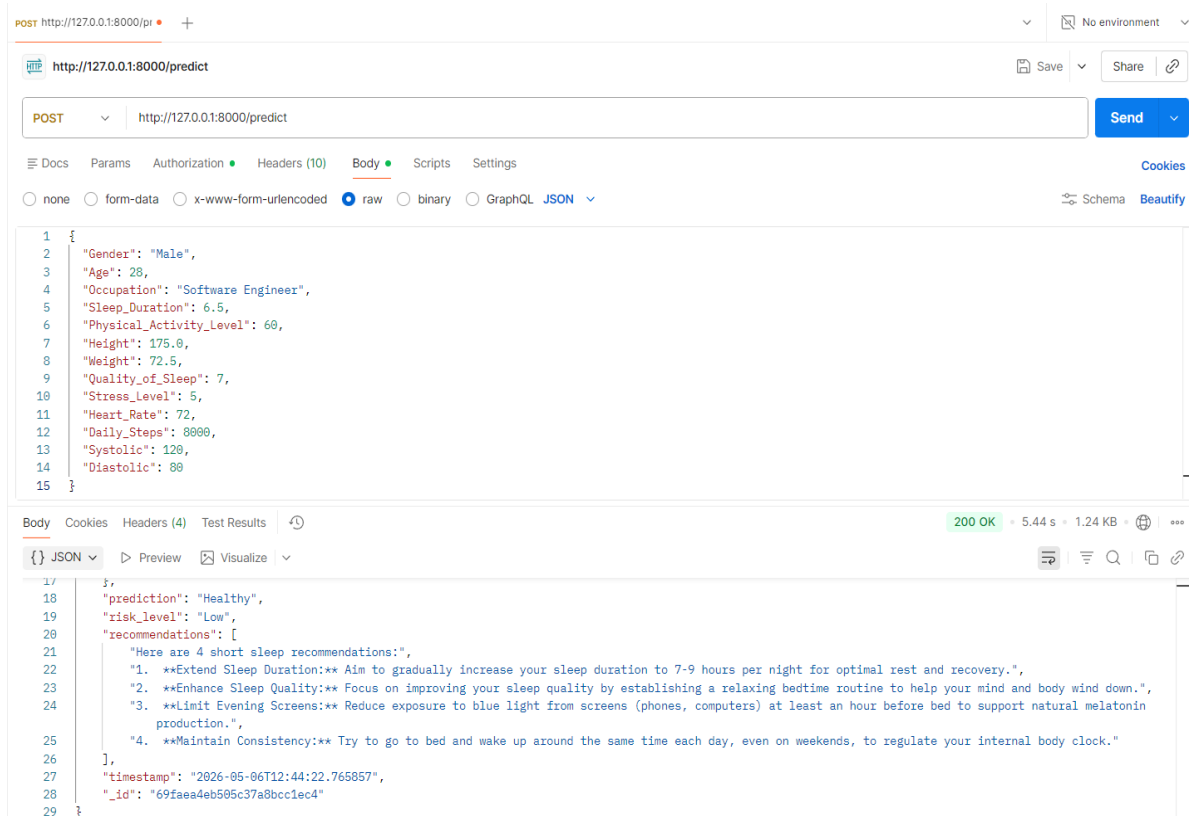


Figure 6.6: Postman API response

6.2.2 Authentication and Authorization

The backend authentication mechanism was implemented using a JSON Web Token (JWT) based approach to maintain secure access control across the system. When login credentials were submitted, the backend validated the provided email and password against the stored records in the database.

Passwords were secured using hashing techniques, ensuring that sensitive user information remained protected at rest. Upon successful authentication, a JWT containing encoded user information and authorization claims was generated and returned to the client. This token was subsequently required in all requests to protected endpoints to verify the identity of the requesting user.

Middleware was incorporated to verify and decode incoming tokens, ensuring that access to restricted functionalities remained limited to authenticated users only. Role-based access control was further enforced to differentiate between standard users and administrators, restricting sensitive operations to authorized personnel.

6.2.3 Recommendation System (LLM Integration)

A recommendation module powered by the Google Gemini API was integrated into the system to provide personalized sleep improvement suggestions based on prediction outcomes and user-submitted data.

A structured prompt engineering approach was employed; whereby relevant user attributes and prediction results were dynamically embedded into predefined prompt templates. These prompts were submitted to the API, and the responses returned were processed and formatted into concise, user-friendly recommendations. This integration extended the system's functionality beyond basic prediction by introducing an intelligent advisory layer.

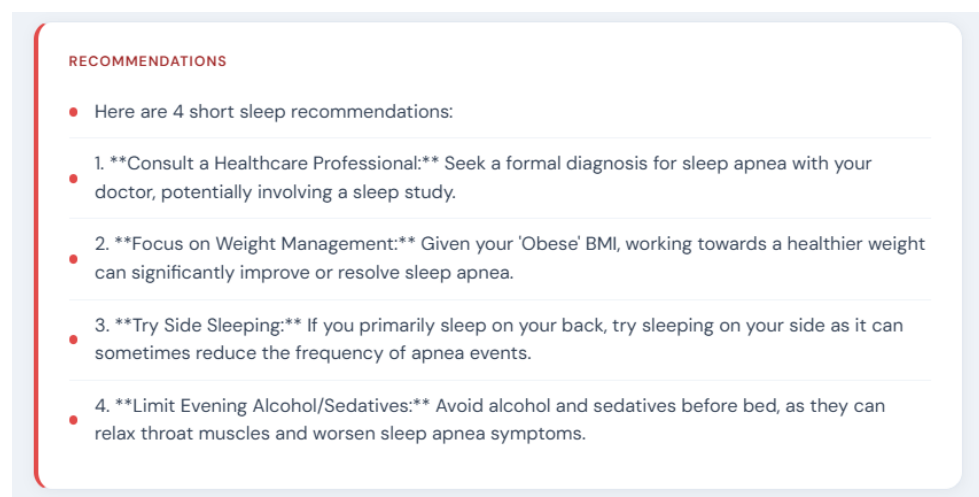


Figure 6.7: Sleep Apnea personalized recommendations

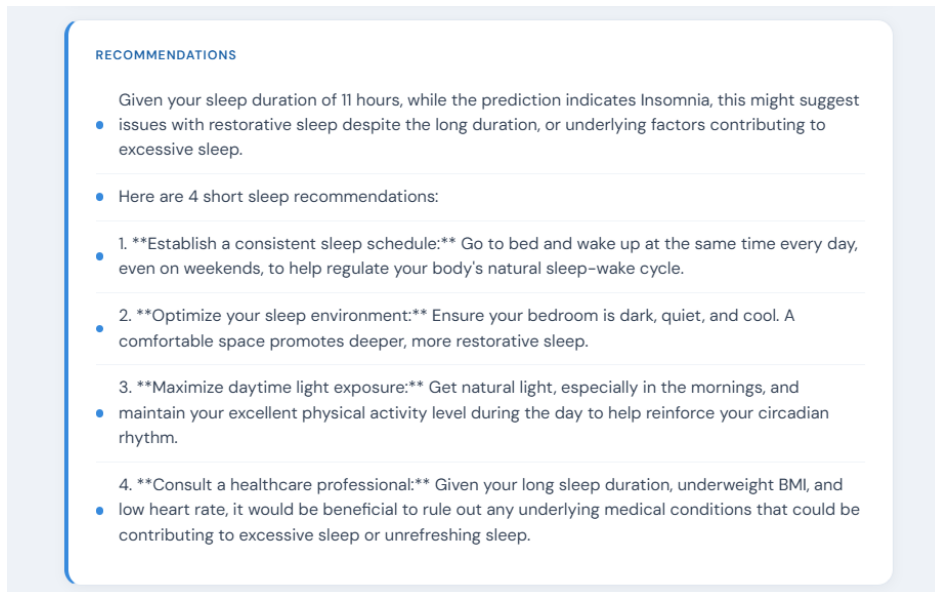


Figure 6.8: Insomnia personalized recommendations

6.2.4 Feature Engineering and Data Enrichment

Prior to passing user input to the machine learning model, the backend carried out a series of preprocessing and feature engineering steps to maintain data consistency and completeness. The key operations performed were:

- Calculation of Body Mass Index (BMI) category derived from the user's height and weight values.
- Handling of optional inputs such as heart rate, daily steps, and blood pressure in cases where values are not provided by the user.
- Application of predefined logic to manage missing or incomplete data entries.
- Generation of additional attributes, including sleep quality and stress level, through rule-based methods.

These steps ensured that the input data conformed to the structure expected by the trained model, improving prediction reliability and consistency.

6.2.5 Database Integration

The system's database operations were handled through MongoDB Atlas, managing user accounts, authentication data, and prediction history records. The collections were structured to support efficient querying and retrieval of user-specific data, allowing users to access their historical predictions and enabling administrators to review system usage patterns over time.

6.2.6 System Workflow and Data Processing

The overall backend workflow followed a structured pipeline. When a user submitted input data through the frontend, it was transmitted to the backend via API request. The backend validated and preprocessed the input, carried out the necessary feature engineering steps, and forwarded the processed data to the machine learning model.

Once a prediction was generated, the result was enriched with personalized recommendations where applicable and stored in the database. The final output was returned to the frontend and presented to the user in a structured and readable format.

6.3 Machine Learning Model Integration

6.3.1 Model Integration

The trained and tuned XGBoost classification model was serialized into a .pkl file using the joblib library, incorporating the trained classifier pipeline and label encoder to ensure all preprocessing steps were preserved alongside the model. The serialized file was loaded into memory during server initialization, eliminating repeated loading overhead and ensuring that prediction requests were handled efficiently at runtime. Once processed input data was received, it was structured into the required format and passed to the model for inference. The complete model training and evaluation notebook along with the dataset used is provided in Appendix 10.7.

```
model_package = {
    "model_name": "Tuned XGBoost",
    "pipeline": best_pipeline,
    "label_encoder": label_enc
}

joblib.dump(
    model_package,
    "/content/drive/MyDrive/Colab Notebooks/Final Project/best_sleep_model.pkl"
)

print("\n Best Model Selected: Tuned XGBoost")
print(" Model + Encoder saved successfully!")
```

```
Best Model Selected: Tuned XGBoost
Model + Encoder saved successfully!
```

Figure 6.9: Model Serialization

6.3.2 Prediction Generation Process

Upon receiving preprocessed input data from the backend, the loaded XGBoost pipeline performed inference and returned a classification result corresponding to one of three categories: Healthy, Insomnia, or Sleep Apnea. The predicted class was then forwarded to the frontend for presentation through the appropriate result interface.

6.4 System Testing

System testing was carried out to verify that all implemented features functioned correctly and that the system behaved as expected under various input conditions. Testing was conducted manually throughout the development process, covering both frontend and backend components. The Microsoft Edge browser was used to access the application on a local development server, and Postman was used independently to validate backend API behavior.

6.4.1 API Testing

The backend API endpoints were tested using Postman to confirm that each responded correctly to both valid and invalid requests. To access protected endpoints, a bearer token was first obtained by submitting valid login credentials to the authentication endpoint. This token was then included in the authorization header of subsequent requests.

Authentication endpoints were tested by submitting both valid and invalid credentials. Valid requests returned the expected token and granted access, whilst invalid credentials produced the appropriate error response. Token verification was further confirmed by deliberately submitting requests to protected endpoints without a valid token; these were correctly rejected by the system.

Prediction endpoints were tested by submitting structured input data and confirming that responses contained the expected classification output with the correct HTTP status code. All endpoints returned consistent, standardized JSON responses, confirming that the API layer functioned reliably.

6.4.2 Functional Testing

Functional testing was performed across all major system modules to confirm that each feature behaved in accordance with the defined requirements.

User registration and login were tested by creating new accounts with valid credentials and subsequently attempting login with both correct and incorrect passwords. Incorrect password submissions produced a clear error message, whilst successful authentication redirected the user to the dashboard as expected.

The password reset functionality was tested by initiating a reset request and verifying that the updated credentials were accepted on subsequent login.

The prediction interface was tested using data rows selected from the training dataset, with records representing each of the three target classes manually entered and submitted individually. In all cases, the system returned the correct predicted class, confirming end-to-end model integration.

The prediction history interface was tested by generating multiple predictions across different categories and verifying that all records appeared correctly. Category filtering was tested by selecting each available filter: Healthy, Insomnia, and Sleep Apnea and confirming that only matching records were displayed. The All tab was verified to display every record regardless of category.

The admin dashboard was tested by logging in with administrator credentials and verifying that the displayed statistics accurately reflected the data held in the database.

Account deletion was tested from the profile management interface, confirming that the account and all associated records were no longer accessible following completion.

6.4.3 Input Validation Testing

Input validation was tested across the prediction form to confirm that the system correctly enforced data constraints and guided users in providing acceptable input.

Required fields were tested by deliberately leaving them empty prior to submission. In each case, an appropriate inline message was displayed prompting the user to complete the missing field. Optional fields were left blank without triggering validation errors, confirming that the system correctly distinguished between required and optional inputs.

Range validation was tested for numerical inputs such as height and weight. When a value below the defined minimum threshold was entered, the form responded with a descriptive message indicating the acceptable range. This confirmed that the validation logic effectively prevented unrealistic values from reaching the prediction model.

6.4.4 Testing Scope and Limitations

All testing was conducted in a local development environment on a desktop machine running Microsoft Edge. The system was not deployed to a live server during the project period, so external user testing was not carried out. The findings therefore reflect system behavior under controlled, single-user conditions. Testing under concurrent multi-user scenarios or across different network environments was outside the scope of this project.

Chapter 7 – End-Project Report

7.1 Project Summary

SleepSense was developed as a web-based sleep disorder prediction system, utilizing machine learning to classify users into one of three categories: no disorder, insomnia, or sleep apnea, based on lifestyle and physiological inputs. The system was implemented as a full-stack application using React, FastAPI, and MongoDB, delivering real-time predictions alongside personalized wellness recommendations through an accessible web interface.

7.2 Evaluation of Project Objectives

- Develop and evaluate a machine learning model for sleep disorder prediction

Met. Multiple classification models were evaluated, and XGBoost was selected as the final classifier, achieving a cross-validated accuracy of 0.81. It is acknowledged that the model was trained exclusively on synthetic data, which constrains its reliability in real clinical settings and represents an honest limitation of the project.

- Collect and preprocess a suitable dataset

Partially met. Data preprocessing was completed successfully; however, reliance on a synthetic dataset rather than clinically validated patient data remains the most significant shortcoming of the project.

- Design and develop a functional web application

Met. A complete full-stack application was developed, and all core functional requirements were implemented and verified through systematic testing.

- Evaluate the system through testing and validation

Partially met. Testing was conducted within a local development environment. The absence of live deployment meant that real-world system performance under concurrent usage and varied network conditions could not be validated.

7.3 Realization of Business Objectives

At prototype level, SleepSense achieved its core goal of providing accessible, non-clinical sleep disorder awareness. The system demonstrated that lifestyle and physiological data could be used to flag disorder risk and prompt users to seek professional medical advice. Full business realization would require live deployment, clinically validated training data, and appropriate regulatory consideration before any application within a public health context could be recommended.

7.4 Changes made during the Project

The only notable change introduced during the project was the replacement of Random Forest with XGBoost as the primary classification model, following comparative evaluation that revealed superior handling of minority classes within the dataset. This decision had no disruption to the overall project timeline. No other significant changes to scope or system design were made.

Chapter 8 – Project post-mortem

In retrospect, the project objectives were appropriate for the scope of a final year project. However, a stronger emphasis on data quality from the outset would have been beneficial. The decision to rely on a synthetic dataset, whilst practical given the constraints of the project, ultimately limited the real-world applicability of the system and represents the most significant area in which the project fell short of its potential.

The product specification was largely well-defined; however, certain aspects such as deployment scope and user authentication were not formally addressed during the initial requirements phase. This introduced some ambiguity during development, and clearer exclusions within the original specification would have sharpened the project's focus.

The Agile iterative methodology proved to be the appropriate choice for this project, providing the flexibility required when the primary classification model was changed mid-project and allowing individual components to be developed and tested incrementally. The selected technologies were well-suited to the project's requirements, and the final model selection was justified by the comparative evaluation results. Were the project to be repeated, exploration of neural network architectures alongside traditional classifiers would be a worthwhile consideration, particularly given improved data availability.

Time management was inconsistent at certain stages of the project, and the complexity of integrating the machine learning pipeline with the web application was underestimated during the planning phase. In future projects, greater time allocation to integration phases and more formal progress tracking against planned milestones would be advisable.

The most significant lesson drawn from this project is that data quality is more consequential than model sophistication. Future development should prioritize the sourcing of clinically validated data and treat deployment as a core project deliverable rather than a deferred consideration.

Chapter 9 – Conclusions

SleepSense successfully delivered a functional web-based sleep disorder prediction system, meeting its core objectives across machine learning development, full-stack implementation, and accessible risk assessment for end users. The system performed at a reasonable level given the constraints of the project and was implemented and tested as intended. Whilst many project objectives were achieved, certain limitations remain, particularly concerning data authenticity and deployment scope. These are acknowledged as directions for future improvement rather than failures of delivery. Future development would benefit from the incorporation of clinically validated training data, live deployment to a production environment, and integration with wearable health monitoring devices. With these enhancements, the system would have the potential to evolve from a functional prototype into a genuinely impactful sleep health awareness tool.

References

Airlangga, G. (2024) 'EVALUATING MACHINE LEARNING MODELS FOR PREDICTING SLEEP DISORDERS IN A LIFESTYLE AND HEALTH DATA CONTEXT', *JIKO (Jurnal Informatika dan Komputer)*, 7(1), pp. 51–57. Available at: <https://doi.org/10.33387/jiko.v7i1.7870>.

Alom, M.S. *et al.* (2025) 'Enhancing Sleep Disorder Diagnosis with a Machine Learning Approach Using Ensemble Neural Networks Sleep Disorder Diagnosis with Ensemble Neural Networks', *Proceedings of the 2024 11th International Conference on Networking, Systems and Security, NSysS 2024*. Association for Computing Machinery, Inc, pp. 48–55. Available at: <https://doi.org/10.1145/3704522.3704533>.

Kumar, Ashutosh *et al.* (no date) 'Sleep Disorder Prediction Using Machine Learning'. Available at: <https://doi.org/10.51583/IJLTEMAS>.

Mostafa Monowar, M. *et al.* (2024) 'Advanced sleep disorder detection using multi-layered ensemble learning and advanced data balancing techniques', *Frontiers in Artificial Intelligence*, 7. Available at: <https://doi.org/10.3389/frai.2024.1506770>.

Ravikumar, L. *et al.* (no date) 'Impact Factor: 8.232', *International Journal of Engineering Technology Research & Management* [Preprint]. Available at: <https://ijetrm.com/IJETRM>.

Sarala, P. *et al.* (no date) *Sleep Disorder Prediction Using Advanced Machine Learning Techniques, International Journal on Science and Technology (IJSAT) IJSAT25012182*.

Yadav, P.K. *et al.* (2023) 'Sleep Disorder Detection using Machine Learning Method', *2nd International Conference on Automation, Computing and Renewable Systems, ICACRS 2023 - Proceedings*. Institute of Electrical and Electronics Engineers Inc., pp. 1530–1532. Available at: <https://doi.org/10.1109/ICACRS58579.2023.10404662>.

Bibliography

Brownlee, J. (2016) *Machine Learning Mastery with Python*. Machine Learning Mastery.

Géron, A. (2019) *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. 2nd edn. Sebastopol: O'Reilly Media.

McKinney, W. (2022) *Python for Data Analysis*. 3rd edn. Sebastopol: O'Reilly Media.

Banks, A. and Porcello, E. (2017) *Learning React: Functional Web Development with React and Redux*. Sebastopol: O'Reilly Media.

Ramírez, S. (2021) *FastAPI Documentation*. Available at: <https://fastapi.tiangolo.com> [Accessed: January 2025].

MongoDB, Inc. (2024) *MongoDB Atlas Documentation*. Available at: <https://www.mongodb.com/docs/atlas> [Accessed: January 2025].

American Academy of Sleep Medicine (2023) *Sleep Education: Understanding Sleep Disorders*. Available at: <https://sleepeducation.org> [Accessed: December 2024].

National Sleep Foundation (2024) *Sleep Health and Sleep Disorders*. Available at: <https://www.thensf.org> [Accessed: December 2024].

Chapter 10 - Appendices

Appendix 10.1 – User Guide

Steps to Set Up the Project:

1. Download the ZIP file from the OneDrive link provided in Appendix 10.2 and extract its contents to a preferred local directory.
2. Ensure Python 3.12 with pip and Node.js 18 LTS or above with npm are installed on the system.
3. Open the project root directory in a terminal or Visual Studio Code terminal.
4. Create a Python virtual environment by entering “python -m venv .venv” in the terminal and press Enter.
5. Activate the virtual environment:
 - For Windows (PowerShell):
`.\.venv\Scripts\Activate.ps1`
 - For Windows (Command Prompt):
`.\.venv\Scripts\activate.bat`
 - For macOS/Linux:
`source .venv/bin/activate`
6. Navigate to the backend directory using “cd backend” in the terminal.
7. Enter “pip install -r requirements.txt” in the terminal within the backend directory and press Enter to install all backend dependencies.
8. Start the backend server by entering “uvicorn app.main:app --reload” in the terminal. The backend will be accessible at <http://127.0.0.1:8000>.
9. Open a separate terminal window and navigate to the frontend directory using “cd frontend”.
10. Enter “npm install” in the terminal within the frontend directory and press Enter to install all frontend dependencies.
11. Enter “npm run dev” to start the frontend server. The frontend application will be accessible at <http://localhost:5173>.

12. Open <http://localhost:5173> in any supported web browser on the system where the project is currently running to access the SleepSense application. Ensure both servers remain active throughout the session.

How to use the System:

Standard User

1. Navigate to <http://localhost:5173> and click Sign Up to create a new account, or log in using existing credentials.
2. Once logged in, the user dashboard will be displayed, providing an overview of prediction activity.
3. To generate a prediction, click the **Start Prediction** button on the dashboard, complete the input form with the required lifestyle and physiological details, and submit the form.
4. The system will process the input and display the predicted sleep disorder classification along with personalized health recommendations.
5. Previously generated predictions can be reviewed and filtered by outcome category through the history page.
6. Account details and logout options are accessible through the profile page.

Administrator

1. Navigate to <http://localhost:5173> and click Admin Login, then authenticate using administrator credentials.
2. The admin dashboard will display system-wide statistics including total registered users and total predictions generated.
3. Registered user accounts and all prediction records across the system can be monitored and managed from the admin panel.

Test Login Credentials:

The following accounts have been configured for demonstration and evaluation purposes:

- Standard User – Email: dinuki@gmail.com, Password: 20030619
- Administrator – Email: admin@gmail.com, Password: admin123

Appendix 10.2 - Project Source Code

Project Source code link:

https://liveplymouthac-my.sharepoint.com/:u:/g/personal/10953721_students_plymouth_ac_uk/IQD5rf5-1sTIS4ZGVr5pQ-b3AZ-MP3zj9r5VKCqRko557cA?e=w0kmBt

Appendix 10.3 - GitHub Repository

GitHub Repository link: <https://github.com/Dinuki19/SleepSense.git>

Appendix 10.4 - Project Initiation Document (PID)

Project Initiation Document (PID) link:

https://liveplymouthac-my.sharepoint.com/:b:/g/personal/10953721_students_plymouth_ac_uk/IQBMYE1cDnx1Q4QXNwwZMEezATKth5cv1yhHRUmlfC7RoZc?e=KltXFZ

Appendix 10.5 - Interim

Interim Report link:

https://liveplymouthac-my.sharepoint.com/:b:/g/personal/10953721_students_plymouth_ac_uk/IQA94uCdTuvSQa7qqbjxLrAQ3KY9on0Q2hHLlDitSXXk8?e=JrSuci

Appendix 10.6 – Record of Supervisory Meetings

Supervisory meetings report link:

https://liveplymouthac-my.sharepoint.com/:b:/g/personal/10953721_students_plymouth_ac_uk/IQCPWnbFv_ynQqGVhjSOjauFAc6OIMaMpba8nJCCOVsTIYo?e=k7o4FT

Appendix 10.7 – Model Training

Model Training notebook and dataset used:

https://liveplymouthac-my.sharepoint.com/:u/g/personal/10953721_students_plymouth_ac_uk/IQCuZCjCjFFTQZfCywYhvRx9Ad1iHBM9wOIUzpsOoU--n7o?e=5WfuhK

Appendix 10.8 – Supplementary Project Materials

This contains supplementary materials compiled to support the content presented in the main body of the report. The appendix is organized into the following sections: the technology stack mapping outlining the tools and frameworks utilized across the system, the dataset features used in the training and evaluation of the machine learning model, key survey questions and critical findings gathered during the requirements elicitation phase, and user interface screenshots of the developed web application. All features, design decisions, and implementation details referenced within these materials have been described and discussed in full within the relevant main chapters.

Appendix 10.8.1 – Technology Stack mapping

Table 10-1 Technologies used

System Component	Technology	Category
User Interface	React.js	Frontend Framework
Navigation	React Router DOM	Frontend Routing
API Communication	Axios	HTTP Client
Data Visualization	Recharts	Charting Library
UI Design	Lucide React	Icon Library
User Alerts	SweetAlert2	Notification Library
Backend Services	FastAPI	Web Framework

System Component	Technology	Category
Server Runtime	Uvicorn	ASGI Server
Programming Language	Python 3.x	Backend Language
Recommendation Engine	Google Gemini API	AI Service
Prediction Model	XGBoost	Machine Learning Algorithm
ML Support	Scikit-learn	ML Library
Data Handling	Pandas	Data Processing
Numerical Operations	NumPy	Numerical Computing
Class Imbalance Handling	Imbalanced-learn	Data Preprocessing
Database	MongoDB Atlas	NoSQL Database
Authentication	JSON Web Token (JWT)	Security Mechanism
Development Environment	Visual Studio Code	Code Editor
Version Control	Git & GitHub	Code Management
Build Tool	Vite	Frontend Tooling
Environment Management	Python venv	Dependency Management

Appendix 10.8.2 – Dataset Features

Table10-2 Dataset Features

Feature Name	Data Type	Description	Value Range / Categories
Gender	Categorical (object)	Gender of the individual	Male, Female
Age	Integer (int64)	Age of the individual in years	18 – 90
Occupation	Categorical (object)	Profession or occupational category	15 occupation categories
Sleep Duration (hours)	Float (float64)	Average daily sleep duration	4.1 – 12.0 hours
Quality of Sleep (scale: 1–10)	Float (float64)	Self-reported sleep quality rating	1.0 – 10.0
Physical Activity Level (minutes/day)	Integer (int64)	Daily physical activity duration	10 – 120 minutes/day
Stress Level (scale: 1–10)	Integer (int64)	Self-reported stress level	1 – 10
BMI Category	Categorical (object)	Body Mass Index classification	Normal, Overweight, Obese, Underweight
Blood Pressure (systolic/diastolic)	Categorical (object)	Blood pressure measurement recorded as systolic/diastolic values	Split into systolic and diastolic during preprocessing
Heart Rate (bpm)	Integer (int64)	Resting heart rate	50 – 100 bpm
Daily Steps	Integer (int64)	Number of daily steps recorded	2,067 – 19,958
Sleep Disorder	Categorical (object)	Target variable representing sleep disorder category	None, Insomnia, Sleep Apnea

Appendix 10.8.3 – Survey

The following figures present the response distributions obtained from the survey conducted during the requirements elicitation phase of this project. The survey was administered via Google Forms and received 120 responses from participants spanning a range of ages and occupations. The results provided contextual evidence supporting the project's problem statement and confirmed that sleep-related concerns were genuinely present within the general population.

Average sleep duration per night (in hours)

120 responses

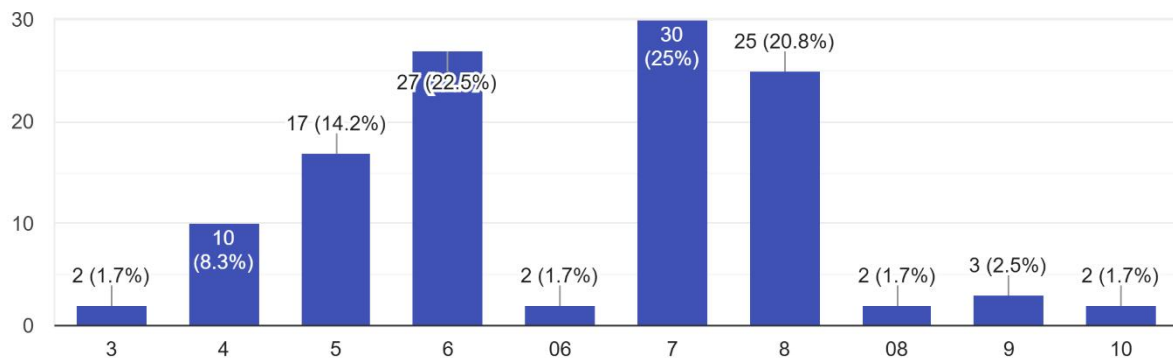


Figure 10.1: Average Sleep Duration Per Night (In Hours)

Many respondents reported sleeping between 6 and 8 hours per night. A total of 24.2% reported sleeping fewer than 6 hours, falling below the clinically recommended minimum of 7 to 9 hours per night.

How often do you wake up during the night?

120 responses

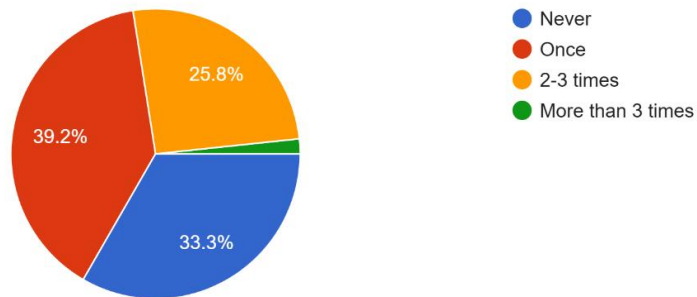


Figure 10.2: Frequency of Night-time Wakeups

A total of 66.7% of respondents experienced at least one night-time wakeup, with 39.2% waking once and 25.8% waking two to three times per night. Only 33.3% reported continuous sleep without interruptions during the night.

Do you feel excessively sleepy or fatigued during the day?

120 responses

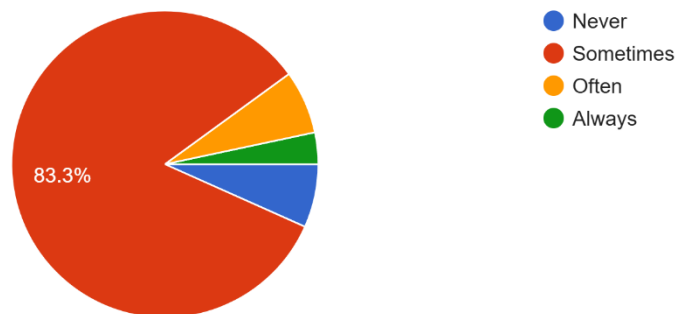


Figure 10.3: Daytime Fatigue Frequency

Many respondents (83.3%) reported feeling excessively sleepy or fatigued during the day sometimes, with additional respondents reporting fatigue often or always. Only a negligible proportion reported never experiencing daytime fatigue, confirming that the impact of disrupted sleep on daily functioning was widely present among the surveyed population.

Do you experience snoring, pauses in breathing, or gasping during sleep?

120 responses

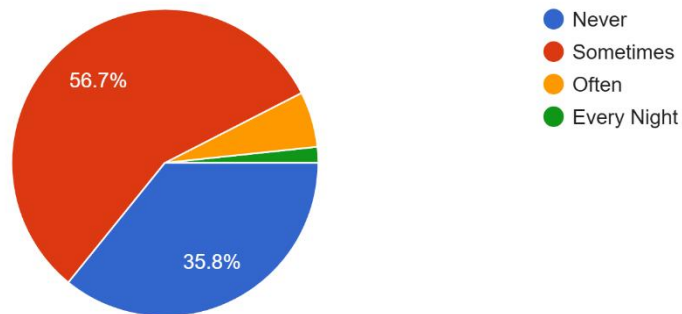


Figure 10.4: Snoring and Breathing Disruptions During Sleep

A total of 64.2% of respondents reported experiencing snoring, breathing pauses, or gasping at least occasionally indicating the presence of potential sleep apnea-related indicators within the surveyed population.

Do you use your phone, tablet, or watch TV right before sleeping?

120 responses

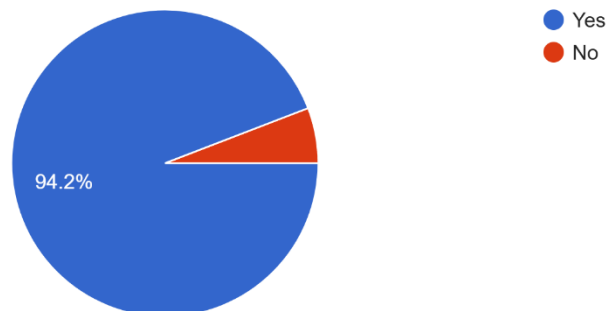


Figure 10.5: Screen Use Before Sleep

A total of 94.2% of respondents reported using a phone, tablet, or watching television immediately before sleeping, highlighting the prevalence of screen exposure as a contributing lifestyle factor to disrupted sleep patterns.

Appendix 10.8.4 – UI screenshots

The landing page served as the public entry point to the SleepSense system, presenting an overview of the platform's purpose alongside navigation options for user registration, standard login, and administrator login. The page also displayed tips on sleep health, warning signs of common sleep disorders, and curated expert articles and resources to promote awareness among visitors prior to registration.



Figure 10.6: Landing page UI

The registration form allowed new users to create a SleepSense account by submitting their personal details. Client-side validation ensured that all required fields were completed correctly before the form was submitted to the backend for account creation.

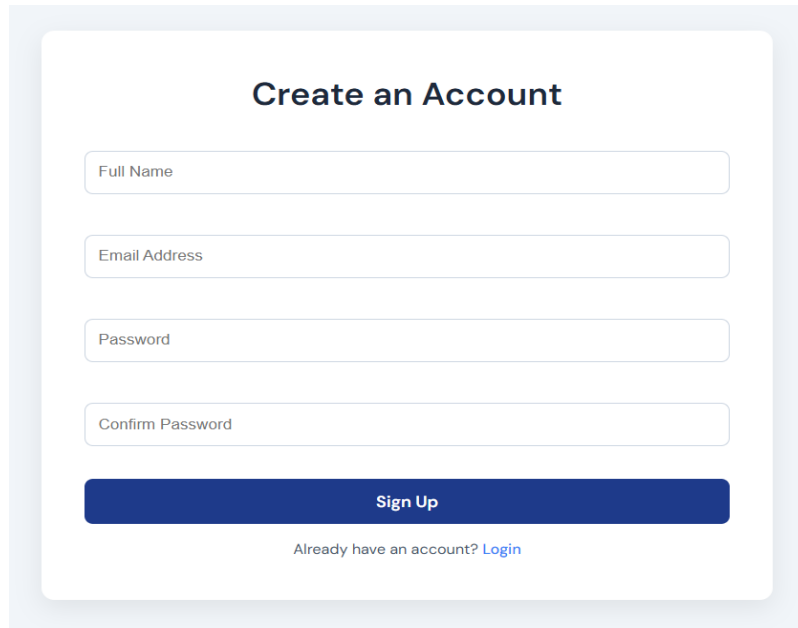
A screenshot of a web form titled "Create an Account". The form is centered on a light blue background. It contains four input fields stacked vertically: "Full Name", "Email Address", "Password", and "Confirm Password". Below these fields is a dark blue button with the text "Sign Up" in white. At the bottom of the form, there is a link that says "Already have an account? [Login](#)".

Figure10.7: User Registration Form

The login interface enabled registered users to authenticate using their email address and password. Upon successful verification, users were redirected to their personal dashboard.

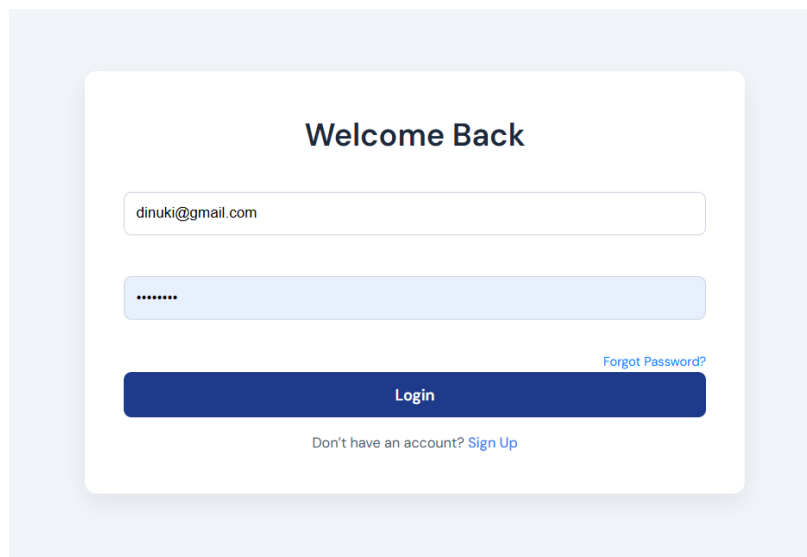
A screenshot of a web form titled "Welcome Back". The form is centered on a light blue background. It contains two input fields stacked vertically: the first contains the email address "dinuki@gmail.com", and the second is a password field with masked characters "*****". To the right of the password field is a link that says "Forgot Password?". Below these fields is a dark blue button with the text "Login" in white. At the bottom of the form, there is a link that says "Don't have an account? [Sign Up](#)".

Figure 10.8: User Login Form

The forgot password interface allowed users to initiate a password reset by entering their registered email address. This feature ensured that users can securely recover access to their accounts when credentials are forgotten.

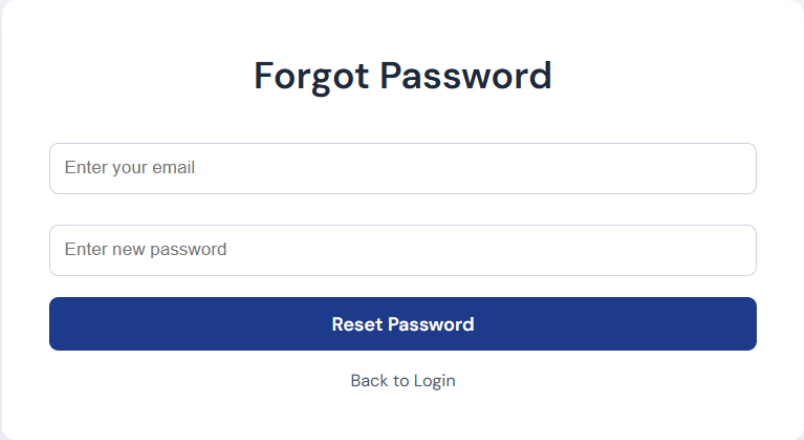
A screenshot of a 'Forgot Password' form. The form is white with rounded corners and is centered on a light blue background. It features a title 'Forgot Password' in bold black text. Below the title are two input fields: 'Enter your email' and 'Enter new password'. A dark blue button labeled 'Reset Password' is positioned below the input fields. At the bottom of the form, there is a link labeled 'Back to Login'.

Figure 10.9: Forgot Password Form

The Insomnia and Healthy result pages followed the same structured layout as the Sleep Apnea result page illustrated in the main body, presenting the predicted condition, a condition overview, relevant health insights, an embedded YouTube video, and personalized recommendations generated by the integrated Gemini API.

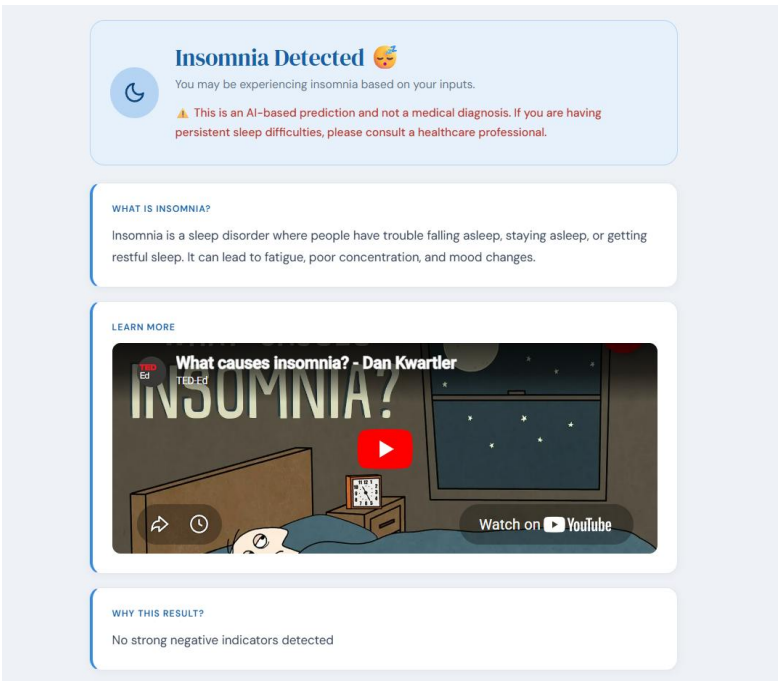
A screenshot of the 'Insomnia Detected' prediction result page. The page has a light blue background. At the top, there is a blue header with the text 'Insomnia Detected' and a sleep icon. Below this, a message states: 'You may be experiencing insomnia based on your inputs.' A warning icon and text follow: 'This is an AI-based prediction and not a medical diagnosis. If you are having persistent sleep difficulties, please consult a healthcare professional.' Below this is a section titled 'WHAT IS INSOMNIA?' with a brief definition. The next section is 'LEARN MORE' and features an embedded YouTube video titled 'What causes insomnia? - Dan Kwartier'. The video thumbnail shows a person in bed with a clock and a window showing a night sky. At the bottom, there is a section titled 'WHY THIS RESULT?' with the text 'No strong negative indicators detected'.

Figure 10.10: Insomnia Prediction Result page

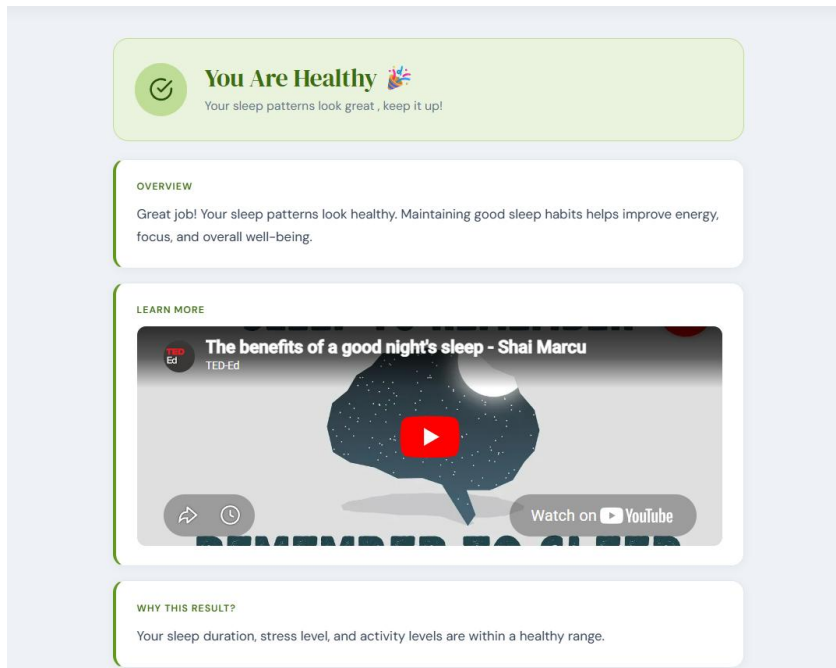


Figure10.11: Healthy Prediction Result Page

The account creation success notification and account deletion confirmation dialog were implemented using SweetAlert2, delivering modal-based feedback for significant user interactions. These notifications ensured that users were clearly informed of the outcome of key account-related actions before and after their completion.

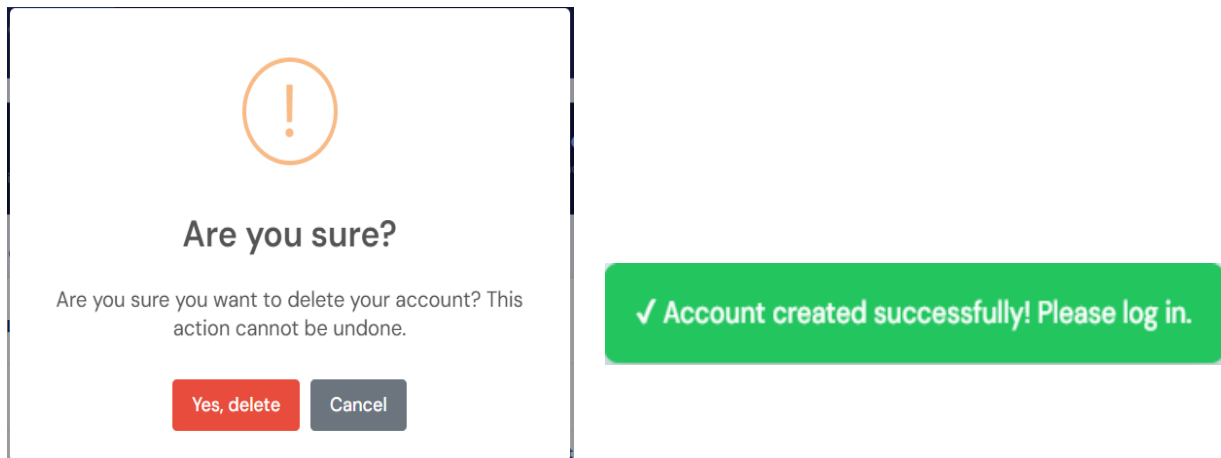


Figure 10.12: System Feedback Notifications

The prediction history page presented a chronological record of all predictions generated by the user, displaying the submitted input data alongside the corresponding outcome for each entry. Category-based filtering options allowed users to view records by specific disorder class, and visual charts summarize outcome distributions over time.

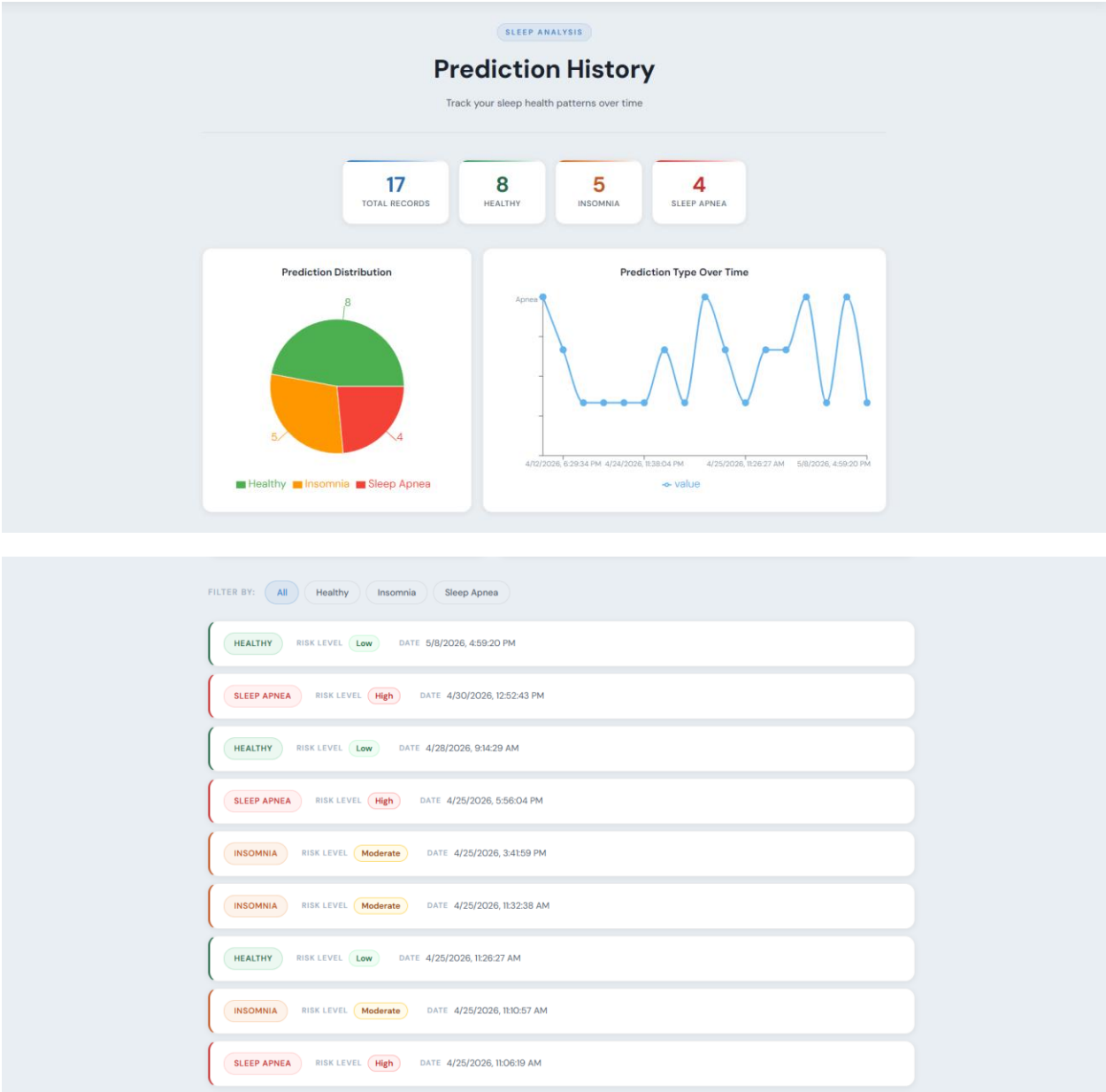
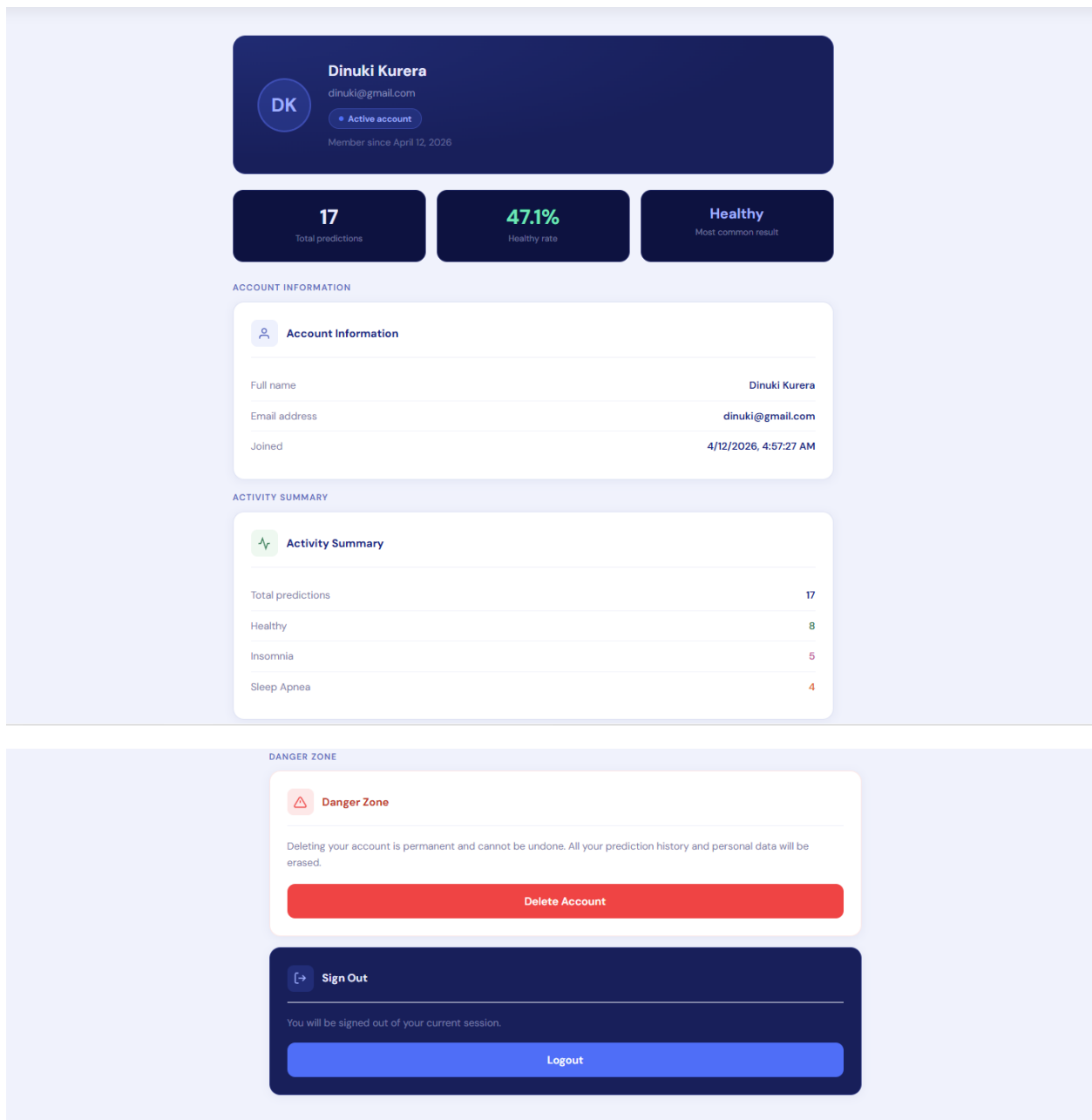


Figure 10.13: Prediction History Page

The profile page displayed account-related information including the registered email address, account creation date, and a summary of system activity. Users can log out or permanently delete their account from this interface.



The image shows a user profile page for 'Dinuki Kurera'. At the top, there's a dark blue header with a circular profile picture placeholder 'DK', the name 'Dinuki Kurera', email 'dinuki@gmail.com', an 'Active account' status, and 'Member since April 12, 2026'. Below this are three summary cards: '17 Total predictions', '47.1% Healthy rate', and 'Healthy Most common result'. The 'ACCOUNT INFORMATION' section includes fields for Full name (Dinuki Kurera), Email address (dinuki@gmail.com), and Joined date (4/12/2026, 4:57:27 AM). The 'ACTIVITY SUMMARY' section lists: Total predictions (17), Healthy (8), Insomnia (5), and Sleep Apnea (4). The 'DANGER ZONE' section contains a warning about account deletion and a red 'Delete Account' button. At the bottom, there's a dark blue 'Sign Out' section with a 'Logout' button.

Dinuki Kurera
dinuki@gmail.com
Active account
Member since April 12, 2026

17
Total predictions

47.1%
Healthy rate

Healthy
Most common result

ACCOUNT INFORMATION

Account Information

Full name: Dinuki Kurera
Email address: dinuki@gmail.com
Joined: 4/12/2026, 4:57:27 AM

ACTIVITY SUMMARY

Activity Summary

Total predictions	17
Healthy	8
Insomnia	5
Sleep Apnea	4

DANGER ZONE

Danger Zone

Deleting your account is permanent and cannot be undone. All your prediction history and personal data will be erased.

Delete Account

Sign Out

You will be signed out of your current session.

Logout

Figure 10.14: User Profile Page

The admin users page provided administrators with a complete list of registered user accounts within the system. Administrators were able to review account details and remove users from the system where necessary.

USER	ROLE	ACTIONS
DK Dinuki Kurera dinuki@gmail.com	User	Delete
NF Nipun Fernando admin@gmail.com	Admin	Delete
VP Vishmi Perera vishmi@gmail.com	User	Delete

Figure 10.15: Admin Panel – Registered Users Overview

The admin predictions page enabled administrators to view all prediction records generated across all user accounts, supporting system-wide monitoring of prediction activity and usage patterns.

USER	PREDICTION	RISK LEVEL
DK Dinuki Kurera	Healthy	Low
DK Dinuki Kurera	Healthy	Low
VP Vishmi Perera	Healthy	Low
VP Vishmi Perera	Sleep Apnea	High
VP Vishmi Perera	Healthy	Low
VP Vishmi Perera	Healthy	Low
VP Vishmi Perera	Insomnia	Moderate
VP Vishmi Perera	Insomnia	Moderate
VP Vishmi Perera	Insomnia	Moderate
VP Vishmi Perera	Healthy	Low
VP Vishmi Perera	Healthy	Low
VP Vishmi Perera	Insomnia	Moderate

Figure 10.16: Admin Panel – User Predictions Overview